

Ecologia de Paisagens com R



Darren Norris

7 de julho de 2026

Índice

Preface

1 Introdução

2 Quantificando a Estrutura da Paisagem

2.1	Apresentação	
2.1.1	Objetivos do Capítulo	
2.2	O Conceito: Iteração, Escala e Interação	
2.3	Preparação: pacotes e dados	
2.3.1	Pacotes	
2.3.2	Dados.	
2.4	Métricas da paisagem e pacote “landscapemetrics”	
2.4.1	Carregar classificação.	
2.4.2	Preparação Pre-processamento	
2.4.3	Função e Processamento	
2.5	Análise e Interpretação	
2.5.1	Diversidade	
2.5.2	Composição	
2.5.3	Configuração	
2.5.4	Correlações entre métricas	
2.6	Implicações para a Fauna	
2.7	Conclusão	

3 Escala de Efeito na Ocorrência de Fauna Doméstica

3.1	Introdução	
3.2	Configuração do Ambiente e Dados	
3.3	Análise da Escala de Efeito	
3.3.1	Autocorrelação Espacial	
3.3.2	Coocorencias	
3.3.3	Escalas diferentes	
3.3.4	Rumo ao Desenho Quase-Experimental	
3.4	Conclusão	

Referências

Preface

Foco em como a Ecologia das Paisagens (com R) nos permite transformar pixels e coordenadas em argumentos sólidos para a conservação e o manejo do mundo real.

1 Introdução

Satélites produzem milhões de pixels todos os dias. Sensores registram a posição de animais com precisão de poucos metros. Mapas de uso da terra estão disponíveis para praticamente qualquer região do planeta. Nunca tivemos tantos dados espaciais. O verdadeiro desafio é transformá-los em evidências científicas robustas e conhecimento ecológico válido. A ecologia da paisagem consiste em transformar representações espaciais em inteligência territorial para orientar a conservação, a restauração e a coexistência em um planeta em mudança.

A maioria dos estudos exige repetir exatamente a mesma análise para dezenas ou centenas de locais de amostragem e em diferentes escalas espaciais. Esse tipo de tarefa rapidamente se torna inviável quando realizado manualmente. Este livro não pretende ensinar as funções disponíveis em R, nem servir como um manual exaustivo de código. Vivemos um momento em que aprender a sintaxe de funções em R deixou de ser o principal desafio: hoje, diversas ferramentas de inteligência artificial ensinam código sob demanda, de forma rápida e personalizada, para praticamente qualquer linguagem ou pacote. Por isso, este livro não pretende ensinar as funções disponíveis em R — esse conhecimento está, literalmente, a um prompt de distância. Em vez disso, procura ensinar uma forma de pensar sobre problemas espaciais: como formular a pergunta certa, escolher a representação de dados adequada e reconhecer qual decisão técnica corresponde a qual raciocínio geográfico ou estatístico. O objetivo é construir fluxos de trabalho claros, reproduzíveis e facilmente adaptáveis a novos conjuntos de dados — algo que nenhuma IA faz sozinha, pois exige compreender o problema antes mesmo de escrever a primeira linha de código.

Este livro foi escrito para estudantes, pesquisadores e profissionais das ciências ambientais que desejam utilizar R como ferramenta para responder perguntas ecológicas reais.

2 Quantificando a Estrutura da Paisagem

 Capítulo em Desenvolvimento

Aviso:

Este capítulo é um trabalho em andamento. Os exemplos de código e o texto ainda estão sendo refinados. Você pode encontrar alguns espaços em branco, rascunhos, erros gramaticais, mas estou trabalhando ativamente para finalizá-lo. Feedbacks e sugestões são sempre bem-vindos!

2.1 Apresentação

Neste capítulo, utilizaremos o Campus Marco Zero da UNIFAP como laboratório de estudo. O foco aqui não é a teoria, mas sim como construir um fluxo de trabalho (workflow) eficiente que permita responder perguntas reais: Qual a escala espacial mais relevante para a caracterização da paisagem que estou estudando? Vamos avançar além do cálculo de métricas em pontos isoladas, como fizemos no Capítulo [Métricas da paisagem](#). Aqui aprenderemos a automatizar a análise de paisagem em múltiplas escalas. Isso permitirá analisar vários pontos em múltiplas escalas simultaneamente.

Para lidar com a complexidade espacial sem nos perdermos tempo em tarefas repetitivas, adotaremos o uso da lógica **Split-Apply-Combine** (Dividir-Aplicar-Combinar) - uma estratégia poderosa para automatizar análises repetitivas. Em vez de calcular métricas manualmente para cada ponto e cada raio de monitoramento, criaremos scripts que automatizam esse processo para locais diferentes e em múltiplas escalas espaciais simultaneamente.

2.1.1 Objetivos do Capítulo

O objetivo é superar o cálculo manual de métricas e implementar fluxos de trabalho automatizados que permitam analisar múltiplos pontos e escalas simultaneamente. Especificamente você aprenderá a:

1. **Quantificar a Paisagem:** Calcular métricas de **Diversidade, Composição e Configuração** através do pacote `landscapemetrics`.
2. **Analisar o Efeito de Escala:** Construir visualizações que revelam como a caracterização da paisagem muda conforme aumentamos o raio de observação ao redor de um ponto.
3. **Interpretar Dados para a Fauna:** Traduzir gráficos de métricas espaciais em conclusões biológicas sobre conectividade e qualidade de habitat para espécies especialistas e generalistas.

i Nota

Embora este capítulo foque na estrutura física, a quantificação correta da paisagem é o primeiro passo para entendermos a **interação biótica**. Por exemplo, não podemos dizer se um cão evita um gato sem antes entender como ambos **interagem com a matriz urbana** ao seu redor.

2.2 O Conceito: Iteração, Escala e Interação

Um dos maiores desafios na ecologia de paisagens é decidir a “escala de efeito”. Em vez de escolhermos um raio arbitrário (ex: 50m), calcularemos métricas em múltiplos raios (12.5m a 100m) para observar como a percepção da paisagem muda para um observador (especie de interesse). Para entender a paisagem, precisamos de pelo menos dois pilares: um técnico (**iteração**) e um teórico (**interação com a escala**).

Nota: É crucial não confundir Iteração (repetição de código) com Interação (relação entre variáveis). Neste capítulo, **iteramos** o código para descobrir como as métricas de paisagem **interagem** com o tamanho do buffer.

2.2.0.1 A Iteração (O “Como fazer”)

Na programação, **iterar** significa repetir uma ação para vários elementos. Em vez de escrever o código 17 vezes (uma para cada ponto (câmera)), criamos um fluxo automatizado. Usaremos o pacote `purrr` para mapear nossas funções sobre os dados, garantindo que a análise seja idêntica para todos os pontos.

Para fazer isso de forma eficiente, adotamos a estratégia Split-Apply-Combine (Dividir-Aplicar-Combinar):

1. **Split:** Dividimos nossa área de estudo em 17 pontos (armadilhas fotográficas).
2. **Apply:** Aplicamos a função de cálculo de métricas (`sample_lsm`) a cada ponto em cada escala.
3. **Combine:** Unimos os resultados em uma única tabela pronta para visualização.

2.2.0.2 A Interação com a Escala (O “Por que fazer”)

A paisagem não é estática; ela muda conforme o “zoom” que utilizamos. A **interação entre escala e percepção** é o que define nossos resultados:

1. **Escala Local (Interação de Micro-habitat):** Em raios pequenos (12.5m), interagimos com a vizinhança imediata. Aqui, a paisagem costuma ser mais homogênea (ou é só floresta, ou é só asfalto).

2. **Escala de Paisagem (Interação de Mosaico):** Em raios maiores (100m), começamos a captar a interação entre diferentes classes de uso do solo. É nesta escala que percebemos o efeito de borda e a fragmentação.

O Duplo Sentido de ‘Mapear’

Na programação funcional, “map” não significa desenhar um mapa geográfico. Significa **associar** cada elemento de um conjunto a uma regra. Quando usamos a função `purrr::map()`, estamos “mapeando” nossas coordenadas geográficas a resultados estatísticos. É o encontro da Álgebra de Mapas de Dana Tomlin (Tomlin 1990) com a Ciência da Computação moderna.

1. O Mapeamento Matemático (A Origem do Código) Na programação e na matemática, “mapear” (*mapping*) não significa desenhar continentes. Significa **associar rigorosamente cada elemento de um conjunto a um elemento de outro conjunto**. Se tem os números [1, 2, 3] e aplica uma regra de “multiplicar por 2”, “mapeou” esses valores para um novo conjunto: [2, 4, 6]. A função `purrr::map()` faz exatamente isso: pega no nosso conjunto de pontos (conjunto A) e aplica uma função a cada uma delas para gerar um novo conjunto de Métricas (conjunto B).

2. O Mapeamento Cartográfico (O Espaço Físico) Na geografia, um “mapa” faz a mesma coisa, mas com o mundo físico. Um cartógrafo “mapeia” o espaço real associando cada árvore, rio ou estrada do mundo tridimensional (conjunto A) a uma coordenada plana de latitude e longitude no papel (conjunto B).

3. O Encontro na Ecologia da Paisagem No nosso script, unimos estes dois mundos. Quando corremos o `purrr::map()`, estamos a **mapear matematicamente o nosso mapa espacial**. Pegamos numa localização no espaço real (o *buffer* de 50 metros em redor de uma armadilha fotografica) e transformamo-la num conceito abstrato: um número que representa a caracterização do espaço (a métrica de paisagem).

Esta mesma lógica é o coração da **Álgebra de Mapas** (criada por Dana Tomlin nos anos 1980 (Tomlin 1990)), que é o que o pacote `terra` usa por trás dos panos. Quando somamos dois *rasters* ou os recortamos, o computador está simplesmente a “mapear” uma regra matemática sobre cada pequeno píxel da imagem, transformando o uso do solo em dados quantitativos.

2.3 Preparação: pacotes e dados

2.3.1 Pacotes

Para reproduzir as análises deste capítulo, você precisará dos seguintes pacotes:

```
# 1. Project Data
library(eprdados)

# 2. Spatial Engines
library(terra)
library(sf)
library(landscapemetrics)
```

```
# 3. Data Manipulation & Grammar
library(tidyverse)
library(broom)

# 4. Analytical Models
library(mgcv)
library(Hmisc)

# 5. Cartography & Visualization
library(tmap)
library(mapview)
```

2.3.2 Dados.

Os dados utilizados nos exemplos de código provêm dos pacotes. Cópias também estão disponíveis. Arquivos espaciais com mapeamento e classificação da paisagem:

- Camadas vetor [marco_zero_2026_31982.gpkg](https://drive.google.com/file/d/1nRJg6WzIjWlBN0ARdn3CU/view?usp=drive_link) https://drive.google.com/file/d/1nRJg6WzIjWlBN0ARdn3CU/view?usp=drive_link
- Camadas raster na pasta [marco_zero_rasters](https://drive.google.com/drive/folders/1vxEg_MDuo99ysbhQ4AU12sNsvWz1MD8s?usp=drive_link) https://drive.google.com/drive/folders/1vxEg_MDuo99ysbhQ4AU12sNsvWz1MD8s?usp=drive_link

```
# Carregando os dados do pacote eprdados
fmz <- system.file("vector/marco_zero_2026_31982.gpkg", package="eprdados")
# load polygon
campus <- st_read(fmz, layer = "campus")
```

```
Reading layer `campus' from data source
  `/home/runner/work/_temp/Library/eprdados/vector/marco_zero_2026_31982.gpkg'
  using driver `GPKG'
Simple feature collection with 1 feature and 7 fields
Geometry type: MULTIPOLYGON
Dimension:      XY
Bounding box:   xmin: 490095.5 ymin: 9998870 xmax: 490890.4 ymax: 10000190
Projected CRS: SIRGAS 2000 / UTM zone 22S
```

```
cameras <- st_read(fmz, layer = "ep_species_pa")
```

```
Reading layer `ep_species_pa' from data source
  `/home/runner/work/_temp/Library/eprdados/vector/marco_zero_2026_31982.gpkg'
  using driver `GPKG'
Simple feature collection with 17 features and 13 fields
```


Geometry type: POINT
Dimension: XY
Bounding box: xmin: 490193.3 ymin: 9998883 xmax: 490793.2 ymax: 9999686
Projected CRS: SIRGAS 2000 / UTM zone 22S

2.3.2.1 Mapa Campus Marco Zero

A figura abaixo apresenta a distribuição espacial das armadilhas fotográficas. (Nota: Uma versão interativa deste mapa está disponível na versão online deste livro [Clique aqui para ver o mapa](#)). Com mapview você pode explorar o mapa interativo abaixo. Você pode utilizar o zoom, arrastar a tela e clicar nos condados para visualizar os valores exatos de cada ponto/polígono.

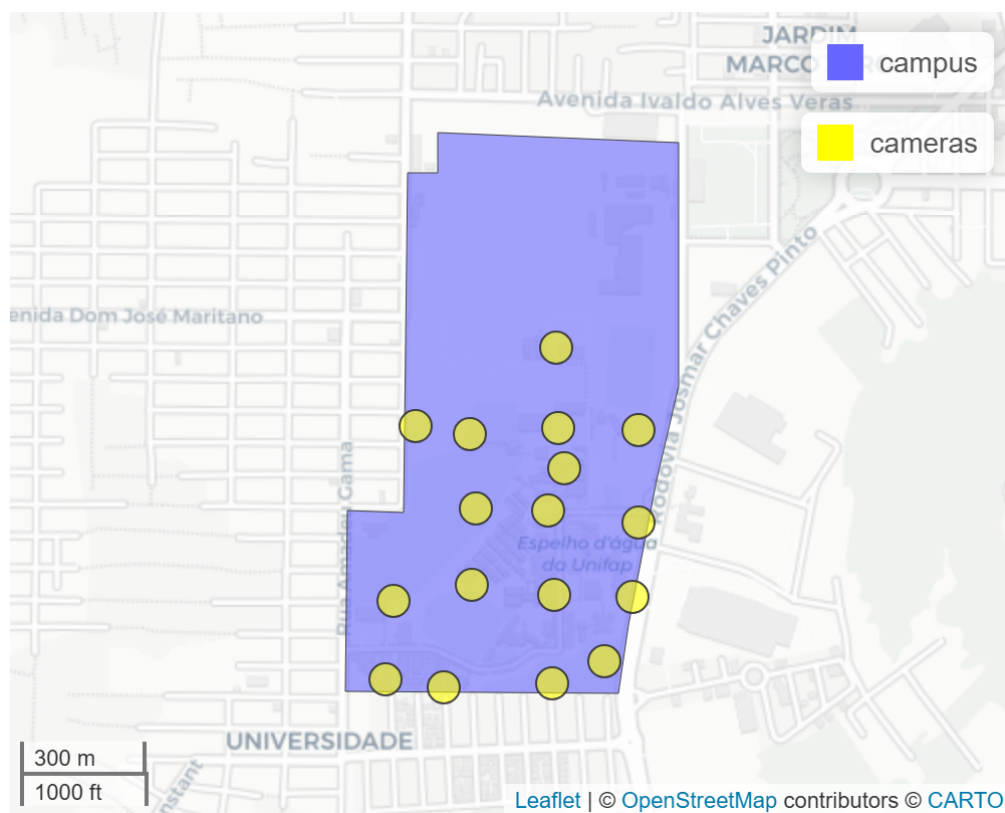


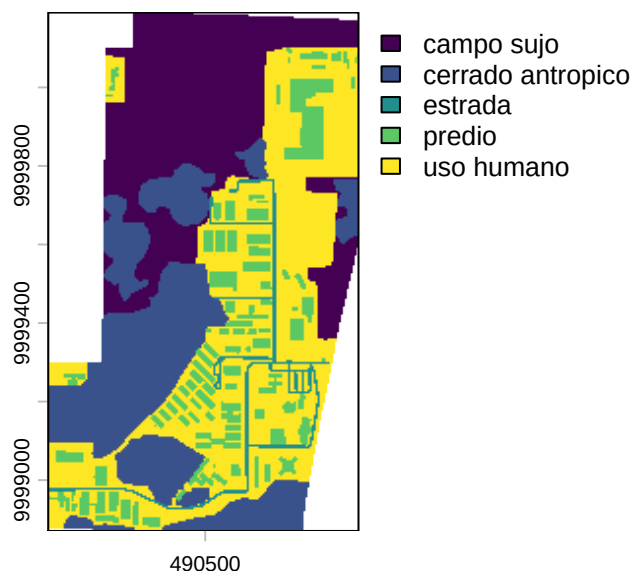
Figura 2.1: Localização das armadilhas fotográficas (cameras) no Campus Marco Zero.

2.4 Métricas da paisagem e pacote “landscapemetrics”

O pacote `landscapemetrics` tem funções para calcular métricas de paisagem em paisagens categóricas (onde tem uma classificação de cobertura de terra/habitat - modelo mancha-corredor-matriz), em um fluxo de trabalho organizado (Hesselbarth et al. 2019). Existem diversos tutoriais e exemplos no site do próprio pacote: <https://r-spatialecology.github.io/landscapemetrics/> .

2.4.1 Carregar classificação.

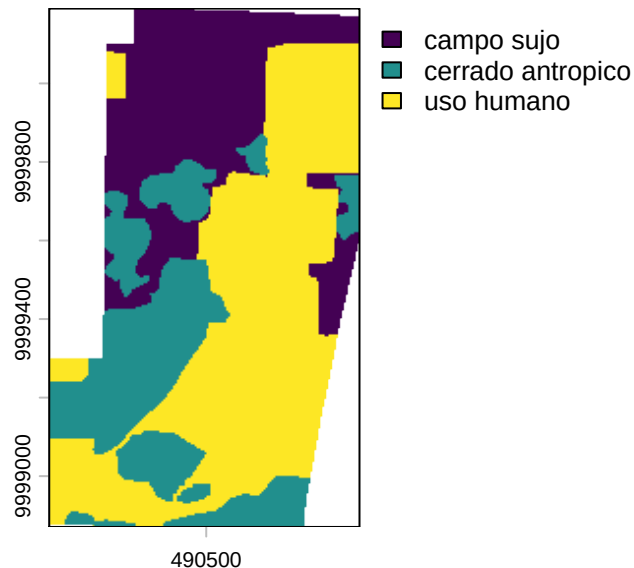
```
class_5 <- rast(class_5)
plot(class_5)
```



```
# Verificamos se o raster é reconhecido pelo landscapemetrics
check_landscape(class_5)
```

```
layer      crs units  class n_classes OK
1         1 projected    m integer         5  v
```

```
class_3 <- rast(class_3)
plot(class_3)
```



```
# Verificamos se o raster é reconhecido pelo landscapemetrics
check_landscape(class_3)
```

```
layer      crs units  class n_classes OK
1         1 projected    m integer      3  v
```

2.4.2 Preparação Pre-processamento

Antes de escrever código para o processamento, precisamos definir e especificar no R alguns valores de referência importantes.

Em vez de utilizarmos um único raio arbitrário, definiremos um vetor com múltiplas escalas de observação:

```
# Definimos os raios (buffers) em metros para escala.
escalas_metros <- c(12.5, 25, 50, 100)
```

Especificar as métricas. A escolha das métricas deve refletir diretamente as perguntas ecológicas. Agruparemos nossos indicadores em três categorias clássicas e, em seguida, os consolidaremos:

```
# métricas desejados
# composição - o que e quanto tem
metrics_comp <- c("lsm_l_ta", "lsm_c_cpland", "lsm_c_pland", "lsm_c_ca")
# configuração - "como" classes são organizados e distribuídos no espaço.
metrics_conf <- c("lsm_c_ed", "lsm_c_ai", "lsm_l_lsi", "lsm_l_pd", "lsm_l_contag")
```

```
# diversidade da paisagem - Quão heterogênea é a matriz paisagística?
metrics_div <- "lsm_l_shdi"

# Consolidando todas as escolhas em um único vetor de consulta
what_metricas <- c(metrics_comp, metrics_conf, metrics_div)
```

2.4.3 Função e Processamento

A verdadeira força de um fluxo de trabalho em R, não está em calcular uma métrica manualmente repetidas vezes, mas em criar funções escaláveis. Vamos encapsular a lógica de extração espacial em uma única função estruturada. No próximo bloco de código criamos a nossa função `processar_uma_camera()`. Nós definimos que ela precisa de exatamente quatro “ingredientes” para funcionar:

`id_alvo`: O número da câmera que será processada agora.

`pontos_totais`: O mapa vetorial com todas as câmeras.

`raster_cheio`: A imagem contendo o uso do solo.

`escalas_alvo`: Os raios de extração (12,5, 25m, 50m, 100m).

```
# Função para processar cada ponto individualmente
processar_uma_camera <- function(id_alvo, pontos_totais, raster_cheio,
                                escalas_alvo) {

  # 1. Isola o ponto de interesse / câmera atual
  ponto_atual <- pontos_totais |> filter(instal_ponto == id_alvo)

  # 2. Recorta o raster para economizar memória (Crop)
  raio_max <- max(escalas_alvo) + 10
  buffer_local <- sf::st_buffer(ponto_atual, dist = raio_max)
  raster_local <- terra::crop(raster_cheio, terra::ext(buffer_local))

  # 3. Itera sobre as escalas (Apply).
  # Calcula as métricas para as três escalas
  # Usamos map_dfr internamente!
  resultados <- purrr::map_dfr(escalas_alvo, function(raio) {

    temp <- landscapemetrics::sample_lsm(
      landscape = raster_local,
      y = ponto_atual,
      plot_id = id_alvo,
      shape = "circle",
      size = raio,
      what = what_metricas
    )
```

```

    temp$escala_buffer <- raio # Adiciona a coluna da escala
    return(temp)

  })

# 4. Faxina (Obrigatório para o purrr não travar o PC)
rm(raster_local, buffer_local, ponto_atual)
gc(verbose = FALSE)
terra::tmpFiles(current = TRUE, remove = TRUE)

# Devolve apenas a tabela com resultados
return(resultados)
}

```

Agora podemos rodar a função com os pontos, raster e raios desejados. Com a nossa função processar_uma_camera pronta, precisamos aplicá-la às nossas 17 armadilhas fotográficas simultaneamente. É aqui que a família map() do pacote purrr brilha, concretizando o paradigma Split-Apply-Combine (Dividir, Aplicar e Combinar) de maneira limpa.

```

# O fluxo para rodar
# (17 pontos x 7 métricas x 4 escalas):
tabela_metricas_final <- purrr::map(
  # A "esteira rolante": O map injeta cada ID no argumento 'id_alvo' sucessivamente
  .x = cameras$instal_ponto,
  .f = processar_uma_camera,
  # Argumentos estáticos repassados identicamente a cada iteração:
  # O 'map' pega esses três dados e os envia exatamente para o
  # 2º, 3º e 4º argumentos da nossa função, sem alterar nada.
  pontos_totais = cameras,
  raster_cheio = class_3,
  escalas_alvo = escalas_metros,
  # barra de progresso
  .progress = "Calculando métricas"
) |>
  # Combinar: Empilhamos todas as tabelas isoladas em um único data frame final
  purrr::list_rbind()

#Incluindo os nomes das classes
tabela_fatores <- cats(class_3)[[1]]

tabela_metricas_final <- tabela_metricas_final |>
  left_join(tabela_fatores, by = c("class" = "value")) |>
  # Reorganizando as colunas para o nome da categoria ficar perto do ID
  relocate(class_3, .after = class)

```

```
# Salvando os resultados de forma segura
if(!dir.exists("data")) dir.create("data", showWarnings = FALSE)
write.csv(tabela_metricas_final,
          "data/tabela_metricas_final.csv", row.names = FALSE)
```

Para processar grandes volumes de dados espaciais sem esgotar a memória do computador, dividimos nossa lógica de iteração entre dois tipos de argumentos:

1. O Argumento Dinâmico (.x):

Ao mapearmos `cameras$instal_ponto`, separamos nossos dados geográficos. O R processa o ambiente de uma única armadilha de cada vez, o que exige um consumo irrisório de RAM com raios de menos de 1 km.

2. Os Argumentos Estáticos:

O modelo digital (raster), o mapa vetorial completo e os raios das escalas são “ferramentas constantes” que permanecem imóveis enquanto o `.x` flui pela função.

O resultado contido em `tabela_metricas_final` está agora no formato tidy (dados arrumados), organizado estruturalmente para fluir diretamente para visualizações com o `ggplot2` ou modelos estatísticos avançados. As colunas mais importantes que vocês precisam compreender são:

- `plot_id`: Identifica qual dos 17 locais está sob análise.
- `class_3`: Descreve a categoria de cobertura de solo referente à linha atual (ex: Área Urbana, Campo Sujo).
- `metric`: Aponta qual métrica foi extraída (ex: `pland`, `shdi`).
- `value`: Armazena o valor quantitativo resultante da métrica.
- `escala_buffer`: Define o tamanho da janela de observação (12.5, 25, 50 ou 100 metros) sob o qual aquele valor foi calculado.

Abordagem `split-apply-combine`. Sempre que precisarem repetir uma análise para dezenas de polígonos, pontos ou imagens, isolem o que identifica o local no `.x`. Todo o resto da base de dados espacial (como os rasters) entra como argumento estático logo abaixo. Isso mantém o código limpo, rápido e mais fácil de ler.

Desafio - O Paradigma Split-Apply-Combine em Ecologia Espacial Antes de escrevermos qualquer linha de código, precisamos entender a estratégia que estamos usando para resolver problemas complexos. Em 2011, foi formalizado no R um conceito chamado Split-Apply-Combine (Dividir, Aplicar e Combinar).

A família `map()` do pacote `purrr` foi projetada especificamente para iterar sobre essas estruturas complexas com muito mais segurança e velocidade. Ao dominar essa lógica, vocês não estão apenas aprendendo a rodar métricas de paisagem; vocês estão ganhando um modelo mental poderoso que pode ser usado para rodar modelos climáticos, processar milhares de arquivos de áudio, ou analisar grandes bancos de metadados genéticos.

A ideia central é simples: quando um desafio analítico é grande demais ou consome muita memória do computador, nós não tentamos resolvê-lo de uma vez só. Nós o quebramos em três etapas lógicas:

1. SPLIT (Dividir) Em vez de tentar processar o raster de uso do solo do estado inteiro de uma vez, nós dividimos o problema.

No nosso código: O argumento `.x = pontos_sf$id_ponto` faz o Split. Ele separa o nosso conjunto de dados de 17 pontos de amostragem e diz ao computador: “Esqueça o todo; olhe apenas para o Ponto 1 agora”.

2. APPLY (Aplicar) Uma vez que o problema foi reduzido a um pedaço gerenciável, nós aplicamos a nossa análise (a nossa função “trabalhadora”) apenas àquele pedaço.

No nosso código: A função `processar_uma_camera` faz o Apply. Ela pega aquele ponto isolado, recorta um mini-raster (crop) ao seu redor e calcula as métricas (`sample_lsm`) para as escalas de 25, 50 e 100 metros. O computador faz isso sorrindo, porque processar um único ponto exige quase zero de memória RAM.

3. COMBINE (Combinar) O R agora tem 17 pequenos resultados flutuando soltos na memória. A etapa final é costurar todos eles de volta em um formato útil.

No nosso código: A função final `purrr::list_rbind()` faz o Combine. Ela recolhe as 17 pequenas tabelas individuais que foram geradas no passo anterior e as empilha perfeitamente em um único Data Frame final, pronto para ser exportado ou visualizado no `ggplot2`.

Entendendo a Mágica do `purrr::map()`: O Dinâmico vs. O Fixo

O desafio é: nós temos 17 câmeras. Como dizemos ao R para rodar essa função 17 vezes sem termos que copiar e colar o código 17 vezes? É aqui que entra o `purrr::map()`. Pensem nele como um gerente de uma linha de montagem inteligente que divide os ingredientes entre o que muda e o que fica fixo.

1. A Esteira Rolante (O Argumento Dinâmico) Observem que dentro do comando `map()`, nós não escrevemos `id_alvo = ...`. Em vez disso, nós passamos a coluna com todos os IDs para o argumento `.x` (`.x = pontos_sf$id_ponto`).

Essa é a regra de ouro do `map`: Ele pega todos os itens que você colocou no `.x` e os entrega, um por um, diretamente no primeiro argumento da sua função. * Na primeira rodada, o `map()` pega a Câmera 1 e injeta no `id_alvo`. A função processa.

Na segunda rodada, ele pega a Câmera 2 e injeta no `id_alvo`.

Ele funciona como uma esteira rolante, alimentando a função até a lista acabar.

2. As Ferramentas Constantes (Os Argumentos Estáticos) Os outros três ingredientes (`pontos_totais`, `raster_cheio` e `escalas_alvo`) não mudam nunca. Não importa se o R está processando a primeira ou a última câmera, a imagem de satélite de fundo e as escalas de 25m, 50m e 100m serão exatamente as mesmas.

Por isso, nós escrevemos esses argumentos com seus nomes exatos na parte de baixo do `map()`. O R entende que eles são ferramentas fixas da bancada de trabalho e os entrega iguazinhos em todas as iterações.

2.5 Análise e Interpretação

2.5.1 Diversidade

```
# Ler o arquivo com metricas
dados_metricas <- read_csv("data/tabela_metricas_final.csv")

# Filtrar os dados
dados_div <- dados_metricas |>
  # Filtrar apenas a métrica de diversidade (shdi)
  filter(metric == "shdi")

grafico_escala_div <- ggplot(dados_div,
                             aes(x = escala_buffer, y = value)) +

  # 1. Pontos Brutos
  geom_jitter(alpha = 0.3, width = 1.5, size = 1.5) +
  # 2. Crossbar
  # Adicionamos aes(group = escala_buffer) LOCALMENTE.
  # Isso avisa ao ggplot: "Para calcular a média e o bootstrap, agrupe por escala.
  # Mas não estrague a continuidade do eixo X para o resto do gráfico".
  stat_summary(
    aes(group = escala_buffer),
    fun.data = "mean_cl_boot",
    geom = "crossbar",
    alpha = 0.4,
    color = "black",
    linewidth = 0.6
  ) +

  # 3. Linha de Tendência (Loess)
  # Como o agrupamento foi isolado no crossbar, o loess consegue enxergar
  # a distância contínua (ex: o salto maior de 50m para 100m) e desenha a curva.
  stat_smooth(
    method = "loess",
    se = FALSE,
    linewidth = 1.2,
    color = "black" # Forçamos a linha para preto para destacar sobre os pontos coloridos
  ) +
  scale_color_viridis_d(option = "turbo") +
```



```

scale_fill_viridis_d(option = "turbo") +
labs(
  title = "Efeito de Escala na Diversidade da Paisagem",
  subtitle = "Variação contínua e tendência (Média ± IC 95%)",
  x = "Raio de Amostragem (metros)",
  y = "Diversidade (Shannon)"
) +
theme_bw() +
theme(
  legend.position = "none",
  strip.text = element_text(face = "bold", size = 11),
  # Aumentar a clareza visual mantendo apenas as linhas principais no eixo X
  panel.grid.minor.x = element_blank()
)

print(grafico_escala_div)

```

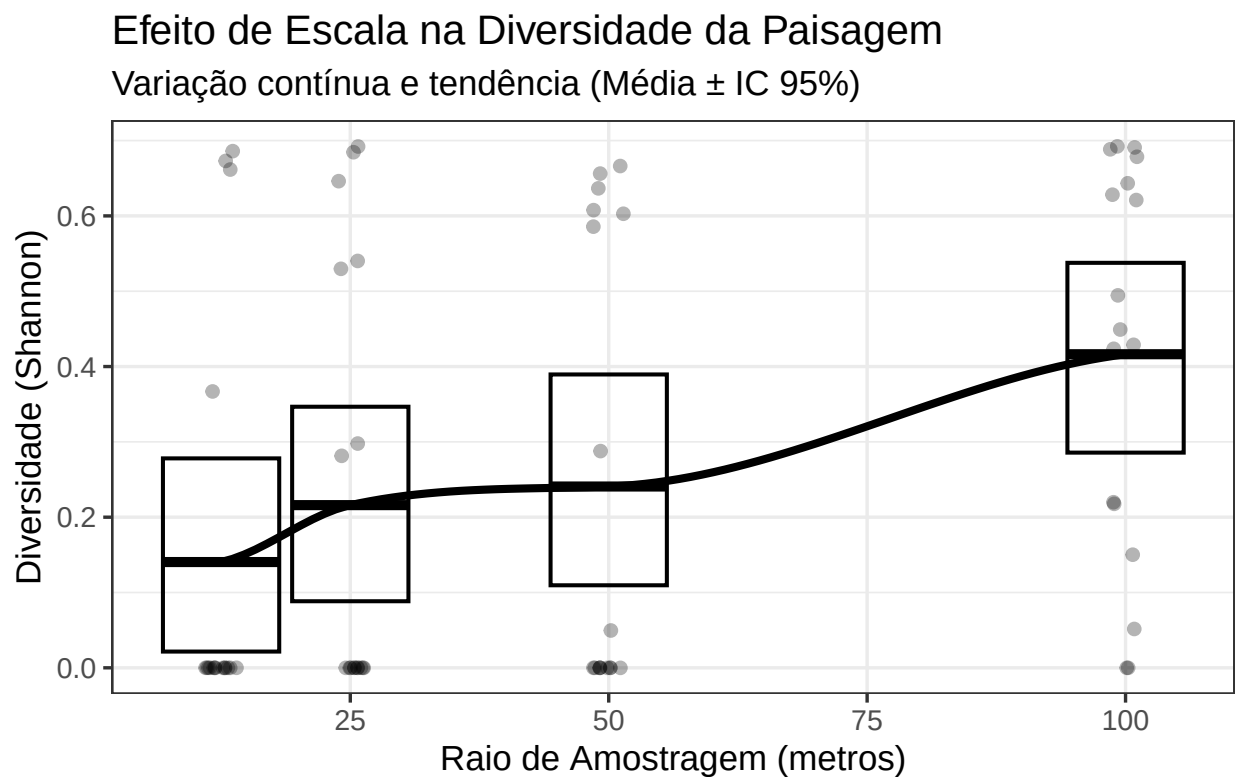


Figura 2.2: Efeito de escala na diversidade da paisagem ao redor das estações de monitoramento fotográfico.

Resultados com o índice de Shannon (métrica SHDI) nos mostra a heterogeneidade da matriz urbana (Figura 2.2). Existe uma tendência positiva clara: a diversidade da paisagem aumenta conforme o

raio de amostragem se expande de 12,5m para 100m (Figura 2.2). É importante notar que, neste caso, o aumento da diversidade não é necessariamente positivo do ponto de vista conservacionista. O aumento do Shannon aqui indica a intrusão de elementos antropogênicos (Uso Humano) na vizinhança da vegetação. A “diversidade” aqui é o registro da mudança entre o ambiente natural e o construído.

O limite superior do intervalo de confiança (IC) para 12,5m termina em aproximadamente 0,28. O limite inferior do IC para 100m começa em aproximadamente 0,30. Como não há sobreposição entre os “crossbars” (IC 95%) desses dois grupos, podemos afirmar com confiança estatística ($p < 0,05$) que a diversidade da paisagem no raio de 100m é significativamente maior que no raio de 12,5m (Figura 2.2). Muitos pontos apresentam SHDI igual a 0 na escala de 12,5m a 50m (Figura 2.2). Isso ocorre porque, em um raio curto, a armadilha fotográfica está inserida em uma “mancha pura” (ex: apenas Cerrado Antrópico ou apenas Uso Humano). A paisagem é homogênea e monótona. Na escala de 100m O valor médio de Shannon sobe para aproximadamente 0,42, com picos atingindo 0,70 (como nos pontos LO_07_0100 e LO_07_0700). Nesta escala, o círculo de amostragem é grande o suficiente para captar a transição entre diferentes classes (ex: o fragmento de vegetação + a calçada).

Os pontos com SHDI alto (perto de 0,7) na escala de 100m são zonas de borda ativa. Estes locais podem atrair animais que utilizam a vegetação para abrigo, mas buscam recursos na matriz urbana (como restos de alimentos humanos ou presas sinantrópicas, como ratos e pombos). A baixa diversidade nas escalas menores (12,5m) confirma que a câmera está “dentro” de um tipo de cobertura. Se um animal é registrado em um ponto onde o Shannon é 0 (escala 12,5m) mas sobe para 0,6 (escala 100m), você tem a prova quantitativa de que aquela espécie está utilizando um fragmento pequeno e altamente influenciado pelo entorno urbano.

2.5.2 Composição

Se o pacote de ecologia espacial simplesmente omite a linha quando a classe não existe no buffer (em vez de reportar 0), a função `mean()` do R calcula a média apenas para os locais onde a classe estava presente. Isto inflaciona artificialmente os valores de `pland` e `cpland`, criando a ilusão de que o campus tem muito mais cobertura do que realmente tem.

```
# Limpar e filtrar os dados
dados_prop <- dados_metricas |>
  # Filtrar apenas a métrica de porcentagem da paisagem (cpland)
  filter(level == "class", metric %in% c("cpland", "pland")) |>
  # Remover linhas que não têm classe definida (ex: métricas de nível de landscape)
  filter(!is.na(class_3)) |>
  tidyr::complete(plot_id, escala_buffer, metric, class_3, fill = list(value = 0))

# Grafico
grafico_escala_efeito <- ggplot(dados_prop,
  aes(x = escala_buffer, y = value,
      color = class_3, fill = class_3)) +

# 1. Pontos Brutos
```

```

geom_jitter(alpha = 0.3, width = 1.5, size = 1.5) +
# 2. Crossbar
# Adicionamos aes(group = escala_buffer) LOCALMENTE.
# Isso avisa ao ggplot: "Para calcular a média e o bootstrap, agrupe por escala.
# Mas não estrague a continuidade do eixo X para o resto do gráfico".
stat_summary(
  aes(group = escala_buffer),
  fun.data = "mean_cl_boot",
  geom = "crossbar",
  alpha = 0.4,
  color = "black",
  linewidth = 0.6
) +

# 3. Linha de Tendência (Loess)
# Como o agrupamento foi isolado no crossbar, o loess consegue enxergar
# a distância contínua (ex: o salto maior de 50m para 100m) e desenha a curva.
stat_smooth(
  method = "loess",
  se = FALSE,
  linewidth = 1.2,
  color = "black" # Forçamos a linha para preto para destacar sobre os pontos coloridos
) +
facet_grid(metric ~ fct_reorder(class_3, value, .fun = mean, .desc = TRUE)) +
scale_color_viridis_d(option = "turbo") +
scale_fill_viridis_d(option = "turbo") +
labs(
  title = "Efeito de Escala na Composição da Paisagem",
  subtitle = "Variação contínua e tendência (Média ± IC 95%)",
  x = "Raio de Amostragem (metros)",
  y = "Proporção na Paisagem (%)"
) +
theme_bw() +
theme(
  legend.position = "none",
  strip.text = element_text(face = "bold", size = 11),
  # Aumentar a clareza visual mantendo apenas as linhas principais no eixo X
  panel.grid.minor.x = element_blank()
)

print(grafico_escala_efeito)

```

Efeito de Escala na Composição da Paisagem

Variação contínua e tendência (Média $\pm$ IC 95%)

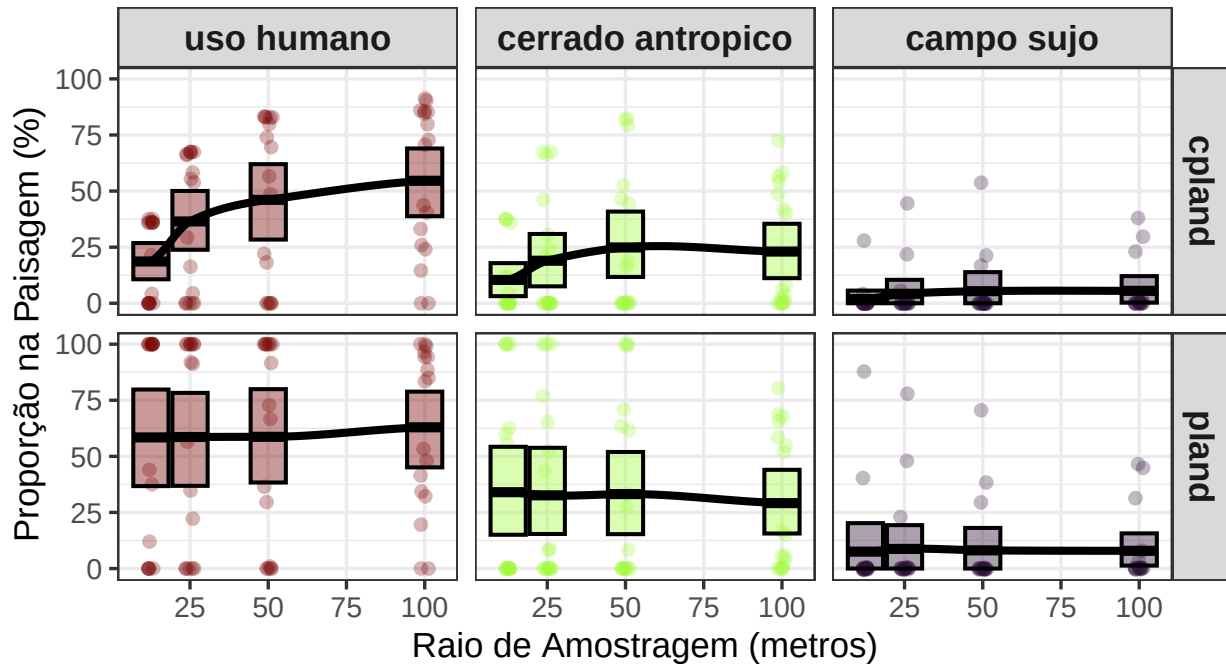


Figura 2.3: Efeito de escala na composição da paisagem ao redor das estações de monitoramento fotográfico. A linha contínua negra representa a tendência espacial suavizada (LOESS) da proporção de cobertura, enquanto as barras indicam a média e o intervalo de confiança de 95% (bootstrap) para os raios de amostragem. Os quadros estão ordenados da classe de uso do solo mais dominante para a menos dominante acerca das estações de monitoramento.

Num ecossistema silvestre intacto, a floresta contínua é a matriz, e as estradas ou clareiras humanas são os fragmentos isolados. Em áreas urbanas inclusive no campus Marco Zero, ocorre exatamente o inverso: o ambiente construído (Uso Humano) é o único espaço contínuo e geometricamente íntegro, enquanto a natureza (Cerrado Antrópico) foi reduzida a recortes altamente permeáveis (Figura 2.3). Isto cria um habitat instável e inadequado para espécies silvestres sensíveis, mas extremamente favorável para a fauna sinantrópica que explora as margens da infraestrutura.

A área núcleo quantificada através a métrica cpland não mede apenas a presença física de uma cobertura (como faz o pland), mas testa a sua integridade geométrica, descontando as zonas de transição (efeito de borda). Os valores de área núcleo do uso humano mantêm-se invariavelmente elevados, registrando médias acima de 50% em raios de 25 metros ou mais (Figura 2.3). Isto prova matematicamente que as infraestruturas e áreas de atividade humana não estão fragmentadas. O uso humano funciona como a “matriz imperturbável” do campus, dominando a paisagem com áreas interiores (núcleo) isoladas de outras classes.

O “Cerrado Antrópico” da UNIFAP, embora presente nos pontos de amostragem, carece de in-

tegridade estrutural. Nas escalas menores (12,5m e 25m), o pland frequentemente atinge valores próximos a 100% (ex: Pontos LO_01_0000 e LO_05_0400). Isso ocorre porque a armadilha está fisicamente dentro da mancha de vegetação. No entanto, ao expandir para a escala de 100m, o pland cai drasticamente para valores como 15,05% (Ponto LO_01_0600) ou 5,74% (Ponto LO_07_0300). Enquanto a área total da classe diminui com a escala, o cpland (Core Area) permanece consistentemente baixo. Em muitos pontos, a área de núcleo é quase nula em relação à complexidade da matriz circundante.

A manutenção de valores baixos de cpland sugere que a maior parte do Cerrado Antrópico no campus é composta por “bordas”. Em ecologia de paisagem, quando o cpland é significativamente menor que o pland, entende-se que o fragmento é excessivamente alongado ou pequeno demais para manter condições de interior (microclima estável, menor ruído, menor incidência de espécies invasoras ou domésticas).

A redução expressiva do pland ao aumentar o raio de amostragem demonstra que as manchas de Cerrado são pequenas e estão imersas em uma matriz dominante de “Uso Humano” (edificações, asfalto, áreas desmatadas). No Campus Marco Zero, isso reflete a pressão da infraestrutura universitária sobre os remanescentes naturais. A paisagem deixa de ser “Cerrado com interferência” para se tornar “Urbanização com fragmentos de Cerrado”.

2.5.3 Configuração

```
# Limpar e filtrar os dados
dados_conf <- dados_metricas |>
  # Filtrar apenas a métrica de porcentagem da paisagem (cpland)
  filter(metric %in% c("ai", "ed")) |>
  # Remover linhas que não têm classe definida
  filter(!is.na(class_3))

# Grafico
grafico_escala_conf <- ggplot(dados_conf,
                              aes(x = escala_buffer, y = value,
                                  color = class_3, fill = class_3)) +

  # 1. Pontos Brutos
  geom_jitter(alpha = 0.3, width = 1.5, size = 1.5) +
  # 2. Crossbar
  # Adicionamos aes(group = escala_buffer) LOCALMENTE.
  # Isso avisa ao ggplot: "Para calcular a média e o bootstrap, agrupe por escala.
  # Mas não estrague a continuidade do eixo X para o resto do gráfico".
  stat_summary(
    aes(group = escala_buffer),
    fun.data = "mean_cl_boot",
    geom = "crossbar",
    alpha = 0.4,
    color = "black",
```

```

    linewidth = 0.6
) +

# 3. Linha de Tendência (Loess)
# Como o agrupamento foi isolado no crossbar, o loess consegue enxergar
# a distância contínua (ex: o salto maior de 50m para 100m) e desenha a curva.
stat_smooth(
  method = "loess",
  se = FALSE,
  linewidth = 1.2,
  color = "black" # Forçamos a linha para preto para destacar sobre os pontos coloridos
) +
facet_grid(metric ~ class_3, scales = "free_y") +
scale_color_viridis_d(option = "turbo") +
scale_fill_viridis_d(option = "turbo") +
labs(
  title = "Efeito de Escala na Configuração da Paisagem",
  subtitle = "Variação contínua e tendência (Média ± IC 95%)",
  x = "Raio de Amostragem (metros)",
  y = "Valores"
) +
theme_bw() +
theme(
  legend.position = "none",
  strip.text = element_text(face = "bold", size = 11),
  # Aumentar a clareza visual mantendo apenas as linhas principais no eixo X
  panel.grid.minor.x = element_blank()
)

print(grafico_escala_conf)

```

Efeito de Escala na Configuração da Paisagem

Variação contínua e tendência (Média $\pm$ IC 95%)

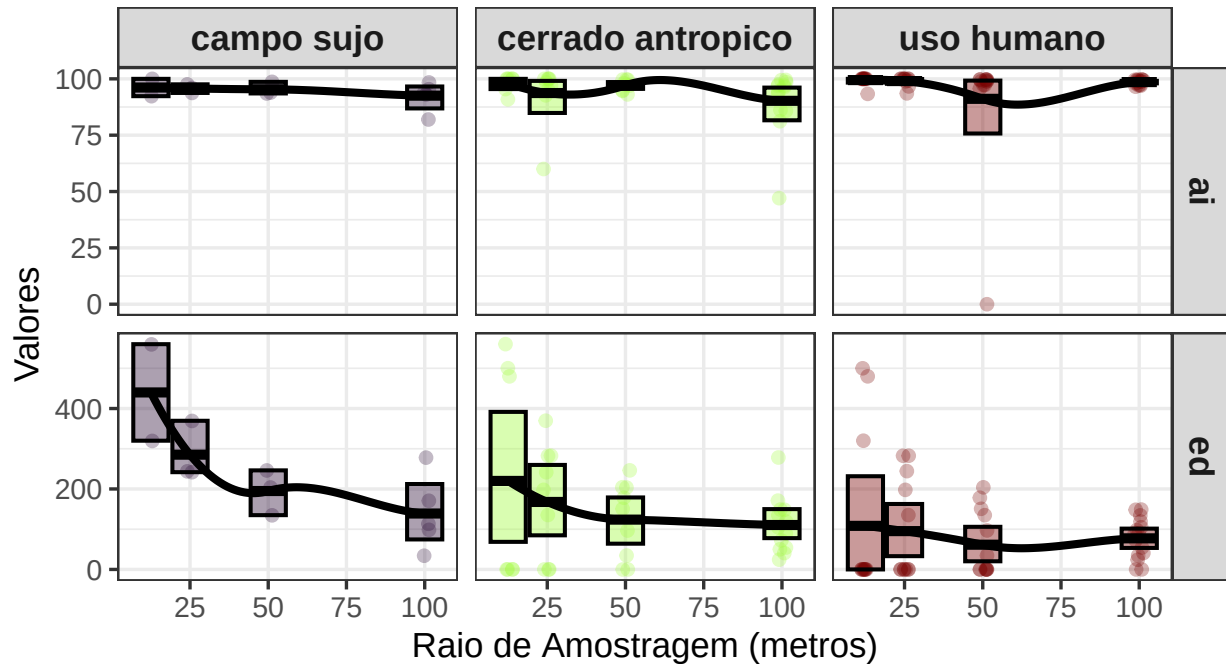


Figura 2.4: Efeito de escala na configuração da paisagem ao redor das estações de monitoramento fotográfico. A linha contínua negra representa a tendência espacial suavizada (LOESS) da proporção de cobertura, enquanto as barras indicam a média e o intervalo de confiança de 95% (bootstrap) para os raios de amostragem. Os quadros estão ordenados da classe de uso do solo mais dominante para a menos dominante acerca das estações de monitoramento.

A configuração da paisagem do Campus Marco Zero (Figura 2.4) revela que os remanescentes de Cerrado e Campo Sujo possuem alta coesão interna (AI 95%), mas sofrem com uma severa pressão de borda nas escalas locais, especialmente o Campo Sujo ($ED > 400$ m/ha). A redução do ED com o aumento da escala espacial demonstra que a complexidade estrutural da vegetação é rapidamente substituída pela homogeneidade da matriz construída, caracterizando um cenário de fragmentação onde os habitats naturais funcionam como ilhas compactas sob forte influência externa.

O índice de agregação (AI) mede o quão próximos ou “agrupados” estão os pixels de uma mesma classe (vai de 0 a 100%). Em todas as classes (Campo Sujo, Cerrado Antrópico e Uso Humano), o AI é extremamente alto e estável (próximo a 100%). Isso indica que a vegetação no campus não está distribuída de forma “salpicada” ou fragmentada em minúsculos pontos isolados (como gramados raleados). Pelo contrário, onde a vegetação existe, ela forma blocos compactos - pelo menos com uma classificação em 3 classes.

O Densidade de Borda (ED) mede a quantidade de bordas (contato entre diferentes classes) em relação à área total (metros por hectare). É um dos principais indicadores de fragmentação. Campo

Sujo (O mais fragmentado), apresenta os valores de ED mais altos na escala local (12,5m), chegando a 400-600 m/ha. Isso mostra que as áreas de Campo Sujo são manchas menores ou com formatos muito irregulares, onde quase tudo é “borda”. Conforme o raio aumenta para 100m, o ED cai para ~150 m/ha, indicando que essas manchas estão imersas em uma matriz que “dilui” essa borda. Para o classe de Uso Humano (a matriz estável) o ED é o mais baixo e estável (perto de 100 m/ha). Isso ocorre porque as áreas construídas no campus (blocos, estacionamentos) tendem a ter formatos geométricos e contínuos, gerando menos “perímetro de borda” por unidade de área do que a vegetação natural, que é mais irregular.

2.5.4 Correlações entre métricas

Para modelos inferenciais (como GLM, GAM etc), a referência na ecologia é: Dormann et al. (2013) - “Collinearity: a review of methods to deal with it and a simulation study evaluating their performance”. Este estudo testou diversos métodos e concluiu que uma correlação entre variáveis de $r > 0.7$ é o ponto onde as estimativas dos coeficientes tornam-se instáveis e o erro do modelo aumenta significativamente. Assim sendo, correlação entre variáveis de $r > 0.7$ se torna um dos critérios objetivos para identificar variáveis redundantes.

i Cuidado com correlação

Misturar métricas de níveis diferentes como nível class com métricas de nível landscape na mesma matriz de correlação é um erro conceitual na modelagem espacial.

Manter as matrizes rigorosamente separadas é o correto pelos seguintes motivos:

Naturezas Espaciais Distintas:

As métricas de classe quantificam o arranjo e a integridade de um habitat específico (ex: a área núcleo do “Cerrado Antrópico”), enquanto as métricas de paisagem avaliam o mosaico como uma entidade única (ex: a textura geral ou a diversidade de manchas no buffer).

Correlações Estruturais Espúrias:

Colocá-las na mesma matriz de correlação frequentemente gera resultados que são apenas artefatos matemáticos. Por exemplo, a proporção da classe mais dominante (pland) quase sempre ditará a diversidade estrutural do buffer inteiro (shdi). Correlacionar essas duas coisas não testa a redundância real dos preditores ecológicos, apenas reflete a estrutura da equação.

O bloco de código abaixo calcula correlações aos pares e gera gráficos para facilitar a comparação das correlações entre as diferentes métricas.

```
# 1. Prepare data
# Filtering for landscape level metrics; pivot so metrics are columns
landscape_wide <- dados_metricas |>
  filter(level == "landscape") |>
  select(plot_id, escala_buffer, class_3, metric, value) |>
  pivot_wider(names_from = metric, values_from = value)

# Filtering for class level metrics; pivot so metrics are columns
```



```

class_wide <- dados_metricas |>
  # Filtrar apenas a métrica de porcentagem da paisagem (cpland)
  filter(metric %in% c("cpland", "pland", "ca")) |>
  # Remover linhas que não têm classe definida (ex: métricas de nível de landscape)
  filter(!is.na(class_3)) |>
  tidyr::complete(plot_id, escala_buffer, metric, class_3, fill = list(value = 0)) |>
  select(plot_id, escala_buffer, class_3, metric, value) |>
  pivot_wider(names_from = metric, values_from = value) |>
  left_join(
    dados_metricas |>
      filter(level == "class", !metric %in% c("cpland", "pland", "ca")) |>
      select(plot_id, escala_buffer, class_3, metric, value) |>
      pivot_wider(names_from = metric, values_from = value)
  )

cor_results_l <- landscape_wide |>
  group_by(escala_buffer) |>
  nest() |>
  mutate(cor_mat = map(data, ~ {
    # Select only numeric columns (metrics)
    numeric_data <- .x |> select(where(is.numeric))

    # Calculate correlation matrix using base R
    # use = "pairwise.complete.obs" handles any NAs safely
    res_matrix <- cor(numeric_data, use = "pairwise.complete.obs")

    # Convert matrix to a tidy data frame
    res_matrix |>
      as.data.frame() |>
      rownames_to_column(var = "metric_name")
  })) |>
  # Remove the raw 'data' column and expand the correlation results
  select(-data) |>
  unnest(cor_mat)
# View the numeric results
print(cor_results_l)

```

```

# A tibble: 8 x 4
# Groups:   escala_buffer [4]
  escala_buffer metric_name    shdi    ta
      <dbl> <chr>          <dbl> <dbl>
1      12.5 shdi         1      0.153
2      12.5 ta          0.153  1
3      25  shdi         1      0.136
4      25  ta          0.136  1

```

5	50	shdi	1	0.0699
6	50	ta	0.0699	1
7	100	shdi	1	0.00994
8	100	ta	0.00994	1

```
cor_results_class <- class_wide |>
  group_by(escala_buffer) |>
  nest() |>
  mutate(cor_mat = map(data, ~ {
    # Select only numeric columns (metrics)
    numeric_data <- .x |> select(where(is.numeric))

    # Calculate correlation matrix using base R
    # use = "pairwise.complete.obs" handles any NAs safely
    res_matrix <- cor(numeric_data, method = "spearman",
                      use = "pairwise.complete.obs")

    # Convert matrix to a tidy data frame
    res_matrix |>
      as.data.frame() |>
      rownames_to_column(var = "metric_name")
  }))) |>
  # Remove the raw 'data' column and expand the correlation results
  select(-data) |>
  unnest(cor_mat)

# View the numeric results
print(cor_results_class)
```

```
# A tibble: 20 x 7
# Groups:   escala_buffer [4]
  escala_buffer metric_name      ca cpland  pland      ai      ed
      <dbl> <chr>      <dbl> <dbl> <dbl> <dbl> <dbl>
1      12.5 ca          1      0.942  0.988  0.450 -0.776
2      12.5 cpland     0.942  1      0.958  0.586 -0.822
3      12.5 pland     0.988  0.958  1      0.544 -0.930
4      12.5 ai        0.450  0.586  0.544  1      -0.727
5      12.5 ed       -0.776 -0.822 -0.930 -0.727  1
6       25 ca          1      0.975  0.994  0.695 -0.716
7       25 cpland     0.975  1      0.974  0.706 -0.754
8       25 pland     0.994  0.974  1      0.670 -0.788
9       25 ai        0.695  0.706  0.670  1      -0.738
10      25 ed       -0.716 -0.754 -0.788 -0.738  1
11      50 ca          1      0.984  0.987  0.965 -0.627
12      50 cpland     0.984  1      0.985  0.961 -0.702
```

13	50	pland	0.987	0.985	1	0.923	-0.727
14	50	ai	0.965	0.961	0.923	1	-0.725
15	50	ed	-0.627	-0.702	-0.727	-0.725	1
16	100	ca	1	0.983	0.983	0.851	-0.0765
17	100	cpland	0.983	1	0.994	0.910	-0.167
18	100	pland	0.983	0.994	1	0.875	-0.139
19	100	ai	0.851	0.910	0.875	1	-0.327
20	100	ed	-0.0765	-0.167	-0.139	-0.327	1

```
# Nothing is normal.
normalidade_por_escala <- class_wide |>
  group_by(escala_buffer) |>
  nest() |>
  mutate(
    # Mapeamos sobre os dados de cada escala
    teste_shapiro = map(data, \(df_escala) {

      df_escala |>
        select(where(is.numeric)) |>
        # Aplicamos o teste para cada coluna numérica
        map(\(coluna) shapiro.test(coluna)) |>
        # O broom::glance transforma o resultado estatístico em uma linha de tabela
        map_dfr(glance, .id = "metrica")

    })
  ) |>
  select(-data) |>
  unnest(teste_shapiro) |>
  # Filtramos para ver rapidamente quais métricas não são normais (p < 0.05)
  mutate(distribuicao = ifelse(p.value < 0.05, "Não-Normal", "Normal"))
```

Visualizar os valores em um gráfico.

```
# 1. Prepare the data for plotting
# We convert the wide columns into a long 'metric_pair' column
cor_plot_data <- cor_results_class |>
  pivot_longer(
    cols = -c(escala_buffer, metric_name),
    names_to = "metric_pair",
    values_to = "r"
  )

# 2. Create the Heatmap Plot
ggplot(cor_plot_data, aes(x = metric_name, y = metric_pair, fill = r)) +
  geom_tile(color = "white") + # White borders for tiles
```

```
# Add numeric labels to each tile for clarity
geom_text(aes(label = round(r, 2)), size = 3, color = "black") +
# Use a diverging color scale (Red = Positive, Blue = Negative)
scale_fill_gradient2(
  low = "#313695", mid = "white", high = "#a50026",
  midpoint = 0, limit = c(-1, 1),
  name = "Spearman\nCorrelation"
) +
# Create a separate panel for each scale
facet_wrap(~escala_buffer, labeller = label_both) +
theme_minimal() +
labs(
  title = "Correlações de métricas da paisagem por escala de buffer",
  subtitle = "Identificação de colinearidade (r > 0,7) para selecionar as métricas finais",
  x = NULL, y = NULL
) +
theme(
  axis.text.x = element_text(angle = 45, vjust = 1, hjust = 1),
  panel.grid = element_blank() # Clean look for heatmaps
) +
coord_fixed() # Makes tiles square
```

Correlações de métricas da paisagem por escala de buffer

Identificação de colinearidade (r > 0,7) para selecionar as métricas finais

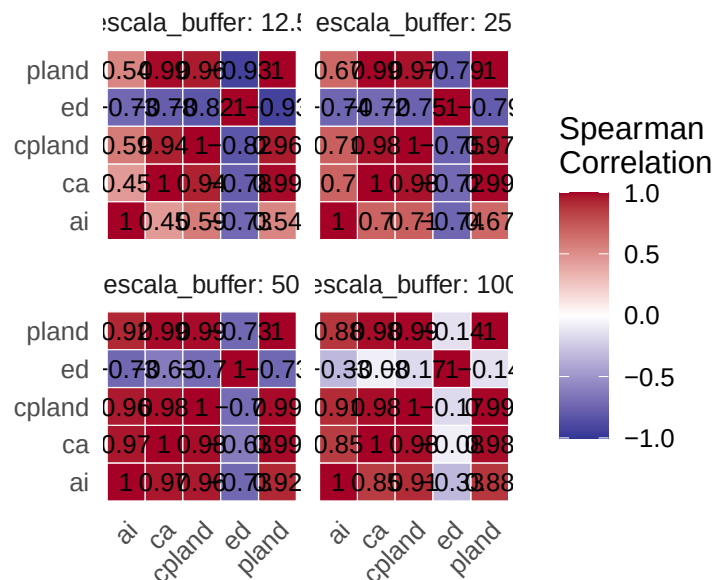


Figura 2.5: A fazer.

Muitas valores nos resultados da Matriz de Correlação (Figura 2.5). Para ajudar indentificar os valores de correlação mais importantes podemos fazer um resumo.

```
# 1. Create the summary table
cor_summary <- cor_plot_data |>
  # Remove self-correlations (e.g., ai vs ai)
  filter(metric_name != metric_pair) |>
  # Ensure "Metric A - Metric B" and "Metric B - Metric A" are treated as the same pair
  mutate(pair = ifelse(metric_name < metric_pair,
                        paste(metric_name, metric_pair, sep = " - "),
                        paste(metric_pair, metric_name, sep = " - "))) |>
  # Group by the pair to calculate stats across all scales (escala_buffer)
  group_by(pair) |>
  dplyr::summarize(
    mean_correlation = mean(r, na.rm = TRUE),
    min_correlation = min(r, na.rm = TRUE),
    max_correlation = max(r, na.rm = TRUE)
  ) |>
  # Keep only unique pairs
  distinct() |>
  mutate(flag_co = ifelse((abs(max_correlation)> 0.699|
                           abs(min_correlation)> 0.699),
                           "colinearidade", "manter"))

print(cor_summary)
```

```
# A tibble: 10 x 5
  pair          mean_correlation min_correlation max_correlation flag_co
  <chr>          <dbl>          <dbl>          <dbl> <chr>
1 ai - ca          0.740          0.450          0.965 colinearidade
2 ai - cpland       0.791          0.586          0.961 colinearidade
3 ai - ed         -0.629         -0.738         -0.327 colinearidade
4 ai - pland        0.753          0.544          0.923 colinearidade
5 ca - cpland       0.971          0.942          0.984 colinearidade
6 ca - ed         -0.549         -0.776         -0.0765 colinearidade
7 ca - pland        0.988          0.983          0.994 colinearidade
8 cpland - ed      -0.611         -0.822         -0.167 colinearidade
9 cpland - pland    0.978          0.958          0.994 colinearidade
10 ed - pland      -0.646         -0.930         -0.139 colinearidade
```

Com base no gráfico de calor (Figura 2.5) e na tabela cor_summary, tem um caso claro de redundância nas métricas.

1. O Grupo Redundante (Área)

As métricas ca (Class Area), pland (Percentage of Landscape) e cpland (Core Area Percentage)

of Landscape) estão altíssimamente correlacionadas entre si em todas as escalas (r entre 0,94 e 0,98). À medida que a área total da classe aumenta, a porcentagem da paisagem e a área de núcleo aumentam quase perfeitamente juntas.

Decisão: Escolha apenas UMA destas. **Recomendação:** Fique com a pland. É a métrica padrão na literatura científica por ser normalizada (0-100%) e facilitar a comparação entre áreas de tamanhos diferentes.

2. A Métrica de Configuração (ed)

O ed (Edge Density) apresenta um comportamento interessante de dependência de escala. Na escala de 12.5m, ele é fortemente correlacionado negativamente com a área ($r=-0.86$). Na escala de 100m, essa correlação cai para quase zero ($r=-0.12$). Em escalas pequenas, a borda e a área estão ligadas de forma simples. Em escalas maiores, a paisagem se torna mais complexa e a densidade de bordas passa a ser independente da quantidade de área.

Decisão: Mantenha o ed. Ele será crucial para capturar a configuração em escalas maiores que a métrica de área (pland) não percebe.

2.5.4.1 Resumo: Quais métricas manter no seu modelo final?

Para evitar problemas de multicolinearidade em análises futuras, eu recomendo manter 3 métricas de class:

- pland (Representa a Quantidade: quanto de habitat existe).
- ed (Representa a Borda: quanta interface/ecotone existe com a matriz).

2.6 Implicações para a Fauna

- Espécies Especialistas:
Provavelmente estão ausentes ou apenas de passagem, pois não encontram “área de núcleo” (cpland) suficiente para estabelecer território ou reprodução segura.
- Espécies Generalistas/Sinantrópicas:
São favorecidas. A baixa área de núcleo e a alta permeabilidade da matriz urbana facilitam a presença de espécies que toleram a presença humana (ex: roedores sinantrópicos).
- Conectividade:
O Cerrado Antrópico no campus funciona mais como stepping stones (pontos de parada) do que como um corredor ecológico funcional, devido à rápida perda de representatividade (pland) em escalas espaciais maiores.

2.7 Conclusão

As análises demonstram que a estrutura da paisagem no Campus Marco Zero é dependente da escala. Abaixo de 50 metros, a variabilidade numérica é insuficiente para modelos ecológicos robustos. O que parece um “bloco de vegetação” em uma análise de curto alcance (12,5m), revela-se uma paisagem multifacetada em escala de 100m. Para a fauna, isso significa que não existem refúgios isolados da influência urbana; todo animal que transita pelo campus está a poucos metros de uma mudança na composição da paisagem.

Para entender os processos ecológicos no campus Marco Zero que dependam minimamente de áreas naturais, uma escala de pelo menos 50 metros é o limite mínimo viável.

Aqui estão os dois motivos técnicos que fundamentam a conclusão:

Estatisticamente viável:

Abaixo de 50 metros (nos raios de 12,5m e 25m), a área núcleo de Cerrado (cpland) para muitas câmaras é pura e simplesmente zero. Estatisticamente, é impossível calcular se um animal prefere a vegetação se a sua variável de habitat for apenas uma coluna cheia de zeros. Ao alargar para 50 metros, o cpland atinge o seu pico de representatividade (cerca de 24,8% de média), introduzindo a variabilidade numérica mínima necessária para os modelos funcionarem.

O Comportamento do Animal:

Uma escala de 50 a 100 metros reflete a realidade de que um animal > 500g dentro do campus precisa de “caminhar mais” ou “olhar mais longe” para encontrar um fragmento seguro. Ele não toma a decisão de forragear com base nos 10 metros ao seu redor (que são puramente borda e perturbação), mas sim avaliando o polígono maior.

Esta escolha de mais de 50 metros é geral e não leva em conta as características dos animais. Se o seu objetivo for modelar a presença de cães errantes, gatos ou pombos (fauna sinantrópica), escalas menores de 25 metros continuam a ser potencialmente explicativas, porque esses animais podem tomar as suas decisões baseados na presença imediata de infraestrutura e calçadas.

3 Escala de Efeito na Ocorrência de Fauna Doméstica

🔥 Capítulo em Desenvolvimento

Aviso:

Este capítulo é um trabalho em andamento. Os exemplos de código e o texto ainda estão sendo refinados. Você pode encontrar alguns espaços em branco, rascunhos, erros gramaticais, mas estou trabalhando ativamente para finalizá-lo. Feedbacks e sugestões são sempre bem-vindos!

3.1 Introdução

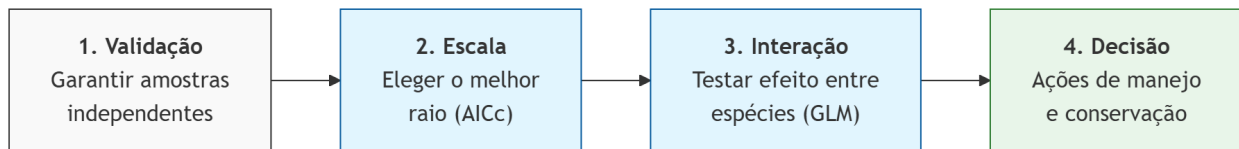


Figura 3.1: Fluxo metodológico para análise multiescalar. O processo inicia na validação estatística da distribuição (1), passa pela identificação da escala de efeito do ambiente (2), isola as interações bióticas entre as espécies (3) e culmina na interpretação para tomada de decisão no mundo real (4).

A ecologia de paisagens urbana investiga como a configuração e a composição do habitat influenciam a biodiversidade. No contexto amazônico, áreas como o Campus Marco Zero da UNIFAP (Macapá, Brasil) representam mosaicos complexos onde a fauna doméstica (*Canis familiaris* e *Felis catus*) interage com fragmentos remanescentes de cerrado.

Um conceito central aqui é a **Escala de Efeito**: a escala espacial na qual a relação entre a estrutura da paisagem e a resposta biológica é mais forte. Este capítulo demonstra como identificar essa escala e analisar a coocorrência dessas espécies usando R.

3.2 Configuração do Ambiente e Dados

Utilizaremos o ecossistema `tidyverse` para manipulação, `sf` para dados espaciais e pacotes de modelagem estatística.


```

# 1. Spatial Engines
library(terra)
library(sf)
library(spdep)

# 2. Data Manipulation & Grammar
library(tidyverse)
library(broom)

# 3. Analytical Models
library(MuMIn) # Para comparação de modelos
library(cooccur)
library(CooccurrenceAffinity)

# 4. Cartography & Visualization
library(patchwork) # Para organizar gráficos

```

Dados

```

# Ler o arquivo com metricas
metricas <- read.csv("data/tabela_metricas_final.csv")
# Ler o arquivo com presença-ausência
especies <- read.csv("data/ep_especies_pa.csv")
# Limpeza e padronização de IDs
especies <- especies |>
  rename(plot_id = instal_ponto)

```

Preparação da Paisagem As métricas calculadas via landscapemetrics precisam ser organizadas. Focaremos na métrica pland (porcentagem da classe) para a classe “uso humano”, que é frequentemente um preditor forte para espécies domésticas.

```

# Filtrando a métrica PLAND para áreas de Uso Humano
df_pland_humano <- metricas |>
  tidyr::complete(plot_id, metric, class_3, escala_buffer, fill = list(value = 0)) |>
  filter(metric == "pland", class_3 == "uso humano") |>
  select(plot_id, escala_buffer, value) |>
  rename(pland_humano = value)

# Preparação: Deixando os dados em formato "longo" (uma coluna para espécie)
# Isso permite agrupar tudo de uma vez só!
df_longo <- especies |>
  select(plot_id, calu, feca) |>
  pivot_longer(cols = c(calu, feca), names_to = "especie", values_to = "presenca") |>
  left_join(df_pland_humano, by = "plot_id")

```

3.3 Análise da Escala de Efeito

3.3.1 Autocorrelação Espacial

A autocorrelação espacial deve ser avaliada primeiro porque ela viola o pressuposto fundamental de independência das observações, o que pode inflar artificialmente a significância estatística (gerando falsos positivos) e distorcer a compreensão dos reais processos que estruturam a paisagem. Para análises precisamos uma objeto com estrutura espacial. O código abaixo transforma uma tabela de dados em arquivo espacial.

```
# Converter para objeto espacial (sf) e projetar para UTM 22S
# Usamos EPSG:31982 (SIRGAS 2000 / zone 22S) para distâncias em metros
dados_sf <- especies |>
  st_as_sf(coords = c("lon_x", "lat_y"), crs = 4326) |>
  st_transform(crs = 31982)
```

Para avaliar o desenho amostral, calculamos a distância até o vizinho mais próximo ($k=1$) utilizando as coordenadas projetadas das armadilhas.

```
# 1. Encontrar o 1º vizinho mais próximo (k = 1)
vizinhos_nn <- knearneigh(st_coordinates(dados_sf), k = 1)

# 2. Converter para objeto de vizinhança (nb)
nb_nn <- knn2nb(vizinhos_nn)

# 3. Extrair as distâncias reais (em metros) entre os pares
# nbdists retorna uma lista, por isso usamos unlist() para ter um vetor numérico
dist_nn <- unlist(nbdists(nb_nn, st_coordinates(dados_sf)))

# 4. Estatísticas
nnd_mean <- mean(dist_nn)
nnd_min <- min(dist_nn)
nnd_max <- max(dist_nn)
```

Em média, cada estação de monitoramento está a 150.5 metros de sua vizinha mais próxima. A menor distância encontrada entre duas estações foi de 96.4 metros, enquanto a maior distância para o primeiro vizinho foi de 191.7 metros.

Do ponto de vista técnico, como a menor distância é de 96.4 m, raios de extração de paisagem acima de 48.2 m resultam em sobreposição (overlap) das áreas medidas, o que deve ser considerado na interpretação dos modelos multiescalares.

Para verificar se a proximidade das armadilhas fotográficas influenciava os registros, aplicamos o teste de Autocorrelação Espacial de Moran I.

```

# Definir a matriz de vizinhança
# Para N pequeno (17), vamos usar KNN (K-Nearest Neighbors,
# com k=3 (cada ponto é comparado aos 3 vizinhos mais próximos)
vizinhos <- knearneigh(st_coordinates(dados_sf), k = 3)
nb <- knn2nb(vizinhos)
pesos <- nb2listw(nb, style = "W") # Estilo 'W' é padronizado por linha

# Executar o Teste de I de Moran para Cães (calu) e Gatos (feca)
moran_caes <- moran.test(especies$calu, pesos)
moran_gatos <- moran.test(especies$feca, pesos)

# Ver os resultados
print(moran_caes)

```

Moran I test under randomisation

```

data:  especies$calu
weights: pesos

```

```

Moran I statistic standard deviate = 0.35438, p-value = 0.3615
alternative hypothesis: greater
sample estimates:
Moran I statistic      Expectation      Variance
    -0.007936508      -0.062500000      0.023706184

```

```
print(moran_gatos)
```

Moran I test under randomisation

```

data:  especies$feca
weights: pesos

```

```

Moran I statistic standard deviate = 1.0157, p-value = 0.1549
alternative hypothesis: greater
sample estimates:
Moran I statistic      Expectation      Variance
    0.10256410      -0.062500000      0.02640892

```

Para ambas as espécies, o p-valor é maior que 0,05 (Cães: p=0,36; Gatos: p=0,15), portanto não podemos rejeitar a hipótese nula de aleatoriedade espacial.

```

# 1. Calcular o lag espacial manualmente e adicionar ao dataframe
dados_plot <- dados_sf %>%
  mutate(
    lag_calu = lag.listw(pesos, calu),
    lag_feca = lag.listw(pesos, feca)
  )

# 2. Criar o gráfico para Cães
p1 <- ggplot(dados_plot, aes(x = calu, y = lag_calu)) +
  geom_jitter(size = 3, width = 0.05, height = 0.05, alpha = 0.6) +
  geom_smooth(method = "lm", color = "black", se = FALSE) + # Reta de Moran
  geom_hline(yintercept = mean(dados_plot$lag_calu), linetype = "dashed", alpha = 0.5) +
  geom_vline(xintercept = mean(dados_plot$calu), linetype = "dashed", alpha = 0.5) +
  coord_cartesian(ylim = c(0, 1.05)) +
  labs(title = "(A) Cães", x = "Ocorrência (Cães)", y = "Lag Espacial") +
  theme_bw()

# 3. Criar o gráfico para Gatos
p2 <- ggplot(dados_plot, aes(x = feca, y = lag_feca)) +
  geom_jitter(size = 3, width = 0.05, height = 0.05, alpha = 0.6) +
  geom_smooth(method = "lm", color = "black", se = FALSE) + # Reta de Moran
  geom_hline(yintercept = mean(dados_plot$lag_feca), linetype = "dashed", alpha = 0.5) +
  geom_vline(xintercept = mean(dados_plot$feca), linetype = "dashed", alpha = 0.5) +
  coord_cartesian(ylim = c(0, 1.05)) +
  labs(title = "(B) Gatos", x = "Ocorrência (Gatos)", y = "Lag Espacial") +
  theme_bw()

# 4. Unir os gráficos com patchwork
p1 + p2

```

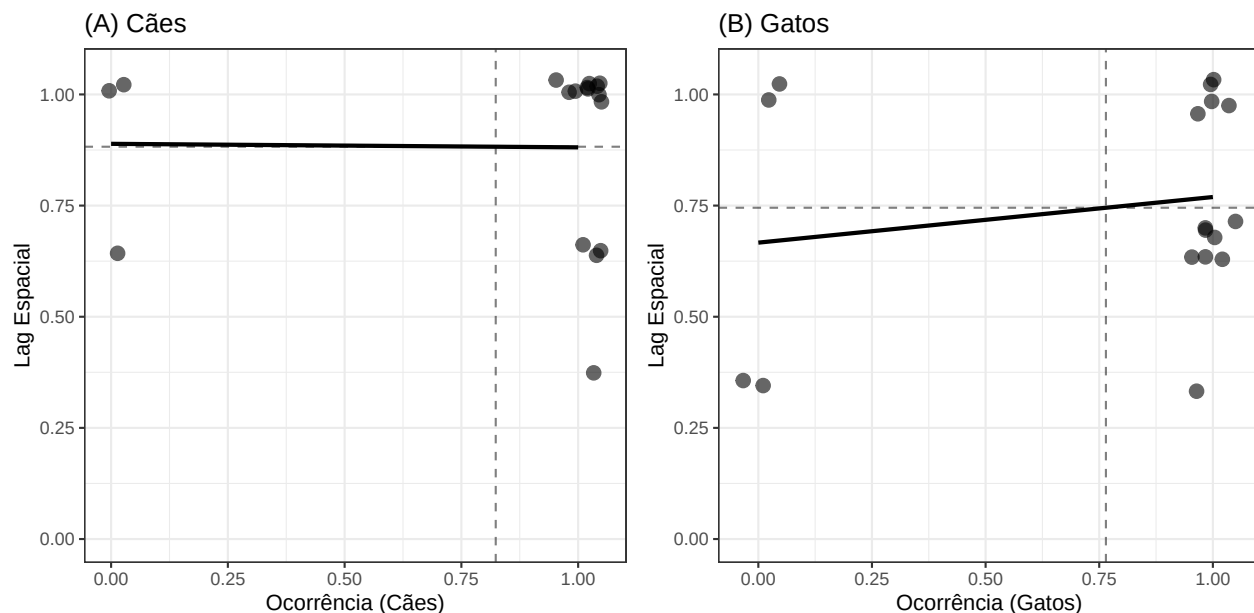


Figura 3.2: Gráfico de dispersão de Moran (Moran Plot) para a ocorrência de cães (A) e gatos (B) no Campus Marco Zero. O eixo horizontal representa os valores observados de presença (1) ou ausência (0) em cada estação de monitoramento. O eixo vertical representa o lag espacial, que corresponde à média ponderada da ocorrência nos três vizinhos mais próximos ($k=3$). A linha sólida representa a inclinação da regressão, cujo coeficiente equivale ao Índice de Moran (I). As linhas tracejadas indicam as médias aritméticas da variável e do seu respectivo lag espacial.

O que isso significa na prática? Isso valida o uso de modelos estatísticos simples (como ANOVA, GLM etc). Como não há autocorrelação espacial significativa, os resíduos dos modelos são independentes, e você não precisa de modelos complexos para “corrigir” a proximidade das câmeras. Para os cães, o Índice de Moran foi próximo de zero ($I=-0,007$; $p=0,36$), o que é visualizado na Figura 3.2 pela linha de regressão praticamente horizontal. Isso indica que a presença de um cão em uma estação não está correlacionada à sua presença nas estações vizinhas. Para os gatos (Figura 3.2), observou-se uma inclinação levemente positiva ($I=0,10$), sugerindo uma tendência sutil de agrupamento, embora estatisticamente não significativa ($p=0,15$).

Para garantir a robustez da nossa análise com 17 estações, utilizamos o teste de Monte Carlo (`moran.mc`) no próximo bloco de código. Em vez de confiar em fórmulas matemáticas ideais (por exemplo o `moran.test` assume que os dados seguem uma distribuição normal, o que raramente é verdade para presença/ausência), este teste embaralha nossos dados 999 vezes para criar um ‘cenário de acaso’ customizado para o Campus Marco Zero.

Ele pega os 17 valores de cães/gatos, mantém a localização das câmeras fixa, e “embaralha” os valores entre elas centenas de vezes (ex: 999 vezes). Para cada embaralhamento, ele calcula o I de Moran. Isso cria uma distribuição nula (o que seria o acaso). O `moran.mc` não retorna um “CI do valor observado”. Ele retorna onde o seu valor observado está em relação a todos os valores possíveis ao acaso. No entanto, você pode calcular os quantis dessa simulação para dizer: “95% das vezes, o

acaso gera valores de Moran entre X e Y". Se o seu valor observado estiver fora desse intervalo, ele é significativo (registros apresentem Autocorrelação Espacial).

```
# Simulação de Monte Carlo (999 permutações)
set.seed(123)
mc_caes <- moran.mc(dados_sf$calu, pesos, nsim = 999)
mc_gatos <- moran.mc(dados_sf$feca, pesos, nsim = 999)

res_caes <- data.frame(moran_i = mc_caes$res,
  obs_i = mc_caes$statistic,
  p_val = mc_caes$p.value
)
res_gatos <- data.frame(moran_i = mc_gatos$res,
  obs_i = mc_gatos$statistic,
  p_val = mc_gatos$p.value
)
# Calcular os quantis de 95% para a distribuição nula
q_caes <- quantile(res_caes$moran_i, probs = c(0.025, 0.975))
q_gatos <- quantile(res_gatos$moran_i, probs = c(0.025, 0.975))
expectativa <- -1 / (17 - 1)

fig_m_cao <- ggplot(res_caes, aes(x = moran_i)) +
  geom_density(fill = "grey90", color = "grey40") +
  # Intervalo de 95% da distribuição nula (Quantis)
  geom_vline(xintercept = q_caes, color = "grey50", linetype = "dotted", linewidth = 0.8) +
  # Linha do valor observado
  geom_vline(aes(xintercept = obs_i), color = "#0072B2",
    linetype = "solid", linewidth = 1.2) +
  # Linha da expectativa (acaso)
  geom_vline(aes(xintercept = expectativa),
    color = "black", linetype = "dashed",
    linewidth = 1.0) +
  annotate("label", x = res_caes$obs_i, y = 1.5,
    label = paste("Observed I:", round(res_caes$obs_i, 3)),
    fill = "white", # Cor do fundo do box
    alpha = 0.7, # Transparência do box (0 a 1)
    label.size = 0.5, # Espessura da borda do box
    vjust = 2,
    color = "#0072B2",
    fontface = "bold", angle = 90, size = 4) +
  labs(title = "Simulação Monte Carlo - Cães",
    subtitle = paste("Permutações: 999 | p-valor:", round(res_caes$p_val, 3)),
    x = "Índice de Moran (I)",
    y = "Densidade") +
  theme_bw()
```

```

fig_m_gato <- ggplot(res_gatos, aes(x = moran_i)) +
  geom_density(fill = "grey90", color = "grey40") +
  # Intervalo de 95% da distribuição nula (Quantis)
  geom_vline(xintercept = q_gatos, color = "grey50", linetype = "dotted", linewidth = 0.8) +
  # Linha do valor observado
  geom_vline(aes(xintercept = obs_i), color = "#D55E00",
    linetype = "solid", linewidth = 1.2) +
  # Linha da expectativa (acaso)
  geom_vline(aes(xintercept = expectativa),
    color = "black", linetype = "dashed",
    linewidth = 1.0) +
  annotate("label", x = res_gatos$obs_i, y = 1.5,
    label = paste("Observed I:", round(res_gatos$obs_i, 3)),
    fill = "white",          # Cor do fundo do box
    alpha = 0.7,             # Transparência do box (0 a 1)
    label.size = 0.5,        # Espessura da borda do box
    vjust = 2,
    color = "#D55E00",
    fontface = "bold", angle = 90, size = 4) +
  labs(title = "Simulação Monte Carlo - Gatos",
    subtitle = paste("Permutações: 999 | p-valor:", round(res_gatos$p_val, 3)),
    x = "Índice de Moran (I)",
    y = "Densidade") +
  theme_bw()

fig_m_cao + fig_m_gato

```

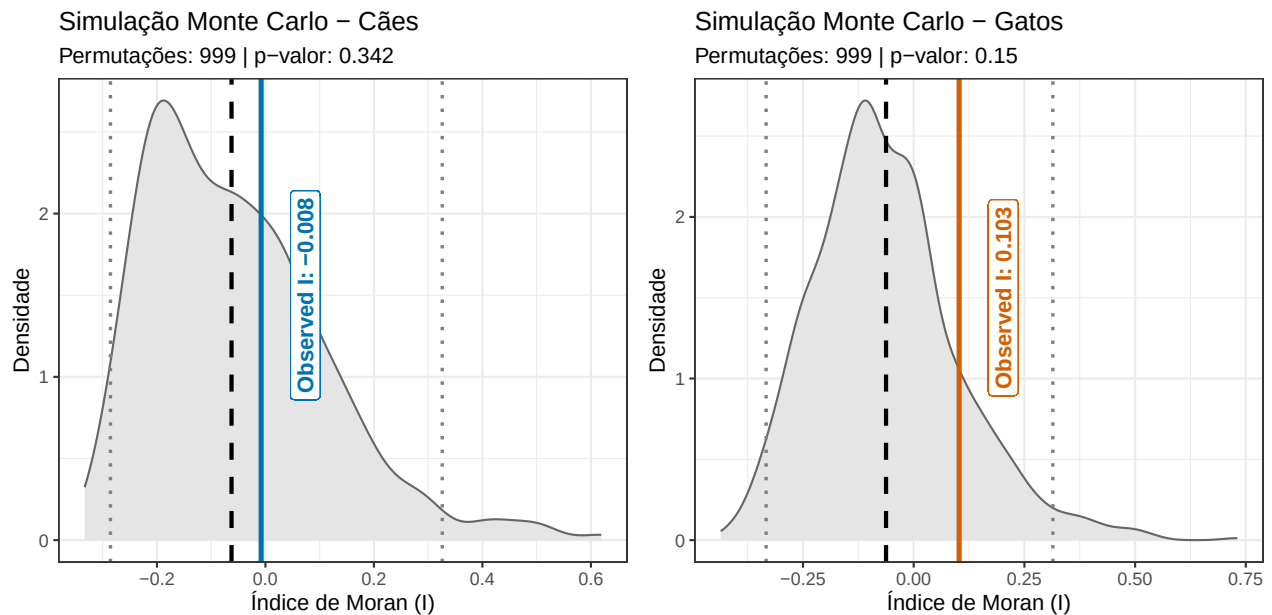


Figura 3.3: Resultados da simulação de Monte Carlo (999 permutações) para o Índice de Moran da ocorrência de cães e gatos no Campus Marco Zero. A área sombreada em cinza representa a distribuição nula do Índice de Moran, ou seja, o padrão esperado caso a distribuição das espécies fosse puramente aleatória no espaço. A linha vertical tracejada indica a expectativa teórica para a ausência de autocorrelação ($I = -0,06$ para $n=17$). As linhas verticais pontilhadas representam quantis de 95% para a distribuição nula.

Na Figura 3.3, a área cinza representa todas as possibilidades de organização espacial que poderiam ocorrer por puro acaso no Campus. Note que tanto para cães quanto para gatos, a linha sólida (nosso dado real) cai dentro dessa zona de densidade cinza. Isso prova visualmente que os nossos registros presença-ausência de fauna não apresentam um padrão espacial de agrupamento ou segregação; eles são indistinguíveis do acaso (padrão aleatório) na escala estudada.

3.3.2 Coocorencias

Agora que sabemos que os 17 pontos podem ser considerados espacialmente independentes, podemos realizar testes estatísticos para a escala do efeito. Aqui, demonstraremos o processo e as etapas utilizando a presença ou ausência de gatos e cães. Primeiramente vamos verificar padrões entre os pontos usando duas técnicas complementares. Inicialmente, utilizamos o modelo probabilístico de coocorrência de espécies desenvolvido por Veech (2013), implementado por meio do pacote *cooccur*. Essa abordagem foi escolhida em detrimento das permutações tradicionais de modelos nulos por fornecer uma solução exata, a qual é mais estável para conjuntos de dados menores ($N=17$), nos quais a aleatorização pode resultar em alta variância. Aplicamos um limiar de filtragem (*thresh* = TRUE) para excluir pares de espécies com poder estatístico insuficiente para atingir significância, garantindo que as classificações como “Aleatório” não fossem meros artefatos decorrentes da raridade. Em segundo lugar, para lidar com o “viés de prevalência” inerente a espécies comuns (por exemplo, *Canis lupus* e *Felis catus*) e para quantificar a magnitude das associações, utilizamos o pacote

Preparação e análises de dados.

	L0_01_0000	L0_01_0200	L0_01_0400	L0_01_0600	L0_03_0000	L0_03_0200
calu	1	0	1	1	1	1
feca	1	1	1	1	1	1
	L0_03_0400	L0_03_0600	L0_05_0000	L0_05_0200	L0_05_0400	L0_07_0100
calu	1	1	1	1	0	1
feca	0	1	1	1	1	1
	L0_07_0300	L0_07_0500	L0_07_0700	L0_09_0300	extra_01_CA	
calu	1	1	0	1	1	
feca	0	0	0	1	1	

Category	Percentage
None	0%
Some	33%
Most	67%

```
|
|
|=====| 100%
```

```
# Summary and significant pairs
summary(cooc_results)
```

Call:

```
cooccur(mat = domestic, type = "spp_site", thresh = TRUE, spp_names = TRUE)
```

Of 1 species pair combinations, 0 pairs (0 %) were removed from the analysis because expected occurrence was < 1 and 1 pairs were analyzed

Cooccurrence Summary:

```

      Species      Sites Positive Negative Random
      2          17          0          0          1
Unclassifiable Non-random (%)
      0          0
attr(,"class")
[1] "summary.cooccur"
```

```
#View P-values and probabilities
prob_table <- prob.table(cooc_results)
prob_table
```

```

  sp1 sp2 sp1_inc sp2_inc obs_cooccur prob_cooccur exp_cooccur  p_lt  p_gt
1   1   2     14     13         11         0.63        10.7 0.87941 0.57941
  sp1_name sp2_name
1    calu    feca
```

```
# Affinity
affinity_res <- affinity(data = domestic, row.or.col = "row")
```

\$all

```

  entity_1 entity_2 entity_1_count_mA entity_2_count_mB obs_cooccur_X total_N
1    calu    feca          14          13          11          17
  p_value exp_cooccur alpha_mle alpha_medianInt conf_level ci_blaker
1      1      10.706    0.567 [-0.268, 1.401]    0.95 [-3.043, 3.418]
      ci_cp      ci_midQ      ci_midP jaccard sorensen simpson
1 [-3.756, 3.846] [-2.391, 3.002] [-3.055, 3.436] 0.688 0.815 0.846
  errornote
1      NA
```

```
$occur_mat
      calu feca
LO_01_0000    1    1
LO_01_0200    0    1
LO_01_0400    1    1
LO_01_0600    1    1
LO_03_0000    1    1
LO_03_0200    1    1
```

```
head(affinity_res$all)
```

```
      entity_1 entity_2 entity_1_count_mA entity_2_count_mB obs_cooccur_X total_N
1      calu      feca              14              13              11          17
  p_value exp_cooccur alpha_mle alpha_medianInt conf_level      ci_blaker
1      1      10.706      0.567 [-0.268, 1.401]      0.95 [-3.043, 3.418]
      ci_cp      ci_midQ      ci_midP jaccard sorensen simpson
1 [-3.756, 3.846] [-2.391, 3.002] [-3.055, 3.436] 0.688 0.815 0.846
  errornote
1      NA
```

Na ecologia, quando duas espécies são muito comuns, elas coocorrem frequentemente simplesmente porque estão em toda parte, e não necessariamente porque “gostam” uma da outra. Os resultados complementares mostraram que no Campus Marco Zero a presença de cães não prediz a presença de gatos, e vice-versa. No 17 pontos do Campus Marco Zero, gatos e cachorros apresentou um padrão de coocorrência aleatório (Modelo Probabilístico de Veech, $p > 0,05$). Embora as espécies tenham compartilhado 11 dos 17 locais, a coocorrência observada (11) não diferiu significativamente dos 10,71 locais esperados ao acaso. Apesar dos elevados índices de similaridade (Jaccard = 0,688), a associação (Alpha MLE = 0,567) não foi estatisticamente significativa, uma vez que o intervalo de confiança de 95% incluiu o valor zero (95% IC: -0,268 a 1,401]; isso sugere que a alta taxa de coocorrência decorre de sua elevada prevalência na área de estudo, e não de uma atração biológica ou de uma exigência de habitat compartilhada.

3.3.3 Escalas diferentes

Os resultados da seção anterior nos trazem uma revelação importante: no Campus Marco Zero, cães e gatos não parecem estar ‘brigando por espaço’ ou se evitando ativamente; a presença de um não prediz a ausência do outro. No entanto, isso não significa que a distribuição deles seja aleatória em relação ao espaço. Se a interação biótica direta (espécie-espécie) é fraca, será que um animal percebe o impacto da urbanização em um raio de 10 metros ou de 100 metros? Para descobrir isso, precisamos investigar a Escala de Efeito. Nesta seção, deixaremos de perguntar ‘quem está com quem’ para perguntar ‘em qual distância o ambiente urbano realmente dita a regra’ para cada espécie.

Para identificar a escala de efeito, testamos a ocorrência da espécie contra a métrica da paisagem em diferentes raios (buffers: 12.5m, 25m, 50m, 100m).

```
# 1. O Fluxo Nest-Map-Unnest (O "Coração" da análise)
# Imagine isso como: Agrupar -> Criar Gavetas -> Rodar Modelo -> Tirar da Gaveta
resultados_modelos <- df_longo |>
  group_by(especie, escala_buffer) |>
  nest() |> # Cria uma "gaveta" (list-column) para cada combinação
  mutate(
    # Aplicamos o GLM dentro de cada gaveta
    modelo = map(data, ~glm(presenca ~ pland_humano, data = .x, family = binomial)),
    # Usamos o broom::glance para extrair estatísticas básicas
    resumo = map(modelo, glance),
    # Adicionamos o AICc manualmente usando o MuMIn
    aicc = map_dbl(modelo, MuMIn::AICc)
  ) |>
  unnest(resumo) |> # Transforma as estatísticas do broom em colunas
  group_by(especie) |>
  mutate(limiar = min(aicc) + 2) |>
  ungroup()
```

Agora comparação dos resultados.

```
resultados_modelos |>
  mutate(especie = ifelse(especie == "calu", "Cães", "Gatos")) |>
  ggplot(aes(x = escala_buffer, y = aicc)) +
  geom_line(aes(color = especie), linewidth = 1.8) +
  geom_point(aes(fill = especie), size = 6, shape = 21) +
  geom_hline(aes(yintercept = limiar), linetype = "dashed") +
  scale_color_manual(values = c("Cães" = "#0072B2", "Gatos" = "#D55E00")) +
  scale_fill_manual(values = c("Cães" = "#0072B2", "Gatos" = "#D55E00")) +
  facet_wrap(~ especie, scales = "free_y") +
  labs(title = "Escala de Efeito", x = "Raio (m)", y = "AICc") +
  theme_bw() +
  theme(legend.position = "none")
```

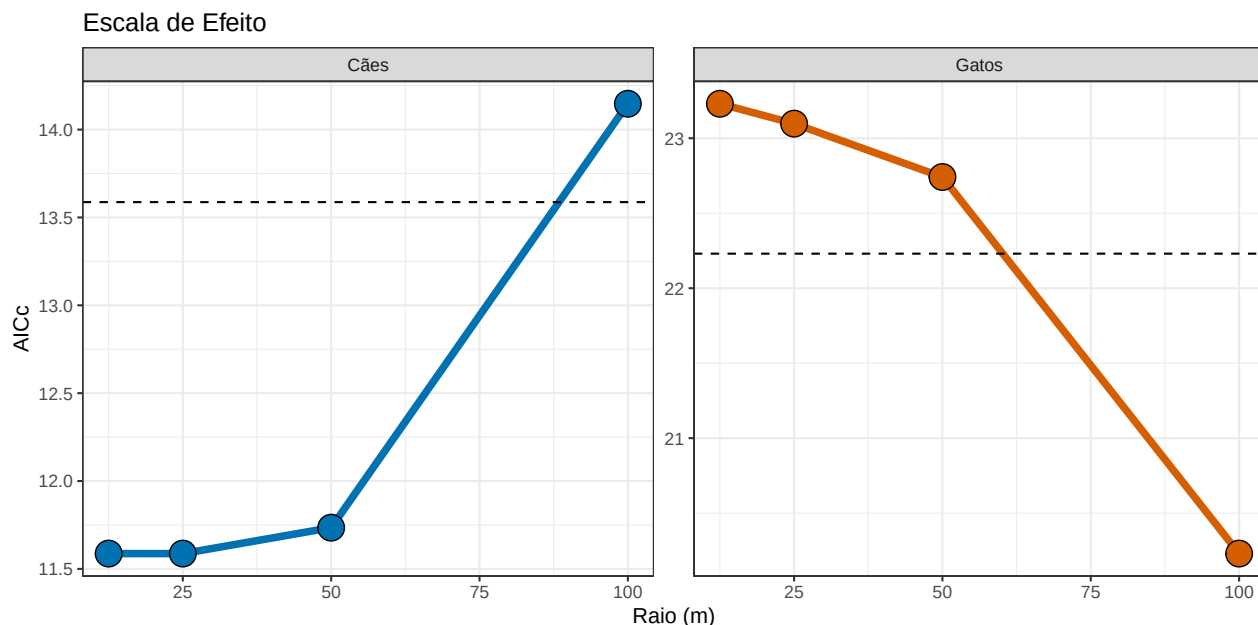


Figura 3.4: Identificação da escala de efeito para a ocorrência de cães (*Canis familiaris*) e gatos (*Felis catus*) em relação à porcentagem de áreas de uso humano (métrica PLAND) no Campus Marco Zero da UNIFAP, Macapá-AP. As curvas representam a variação do Critério de Informação de Akaike corrigido (AICc) em quatro escalas espaciais (raios de 12.5, 25, 50 e 100 metros). A linha horizontal tracejada indica o limiar de parcimônia ($\Delta AICc$ 2); modelos abaixo desta linha são considerados estatisticamente equivalentes em termos das informações que contêm.

Observa-se na Figura 3.4 que a ocorrência de cães é melhor explicada por processos locais (12.5 a 50 m), enquanto a ocorrência de gatos responde de forma mais forte à paisagem em uma escala mais ampla (100 m). Lembre-se de que o AIC compara modelos, não diz se o melhor modelo é válido ou se as tendências são relevantes.

Uma vez identificada a escala de efeito — ou seja, a distância na qual cães e gatos melhor respondem à pressão urbana (Figura 3.4) — torna-se fundamental questionar se essa distribuição é moldada exclusivamente pelo ambiente físico ou se as espécies influenciam diretamente a presença uma da outra. Na ecologia de paisagens urbanas, o habitat pode ser favorável, mas a presença de um potencial competidor ou predador pode atuar como um filtro adicional para a ocorrência de uma espécie. Para investigar isso, deixamos de olhar apenas para a relação ‘indivíduo-paisagem’ e passamos a analisar a ‘interação biótica multiescalar’. No próximo bloco de código, exploramos se a presença de cães exerce algum efeito de atração ou repulsão sobre a ocorrência de gatos em cada um dos raios estudados, permitindo-nos entender se a coexistência no Campus Marco Zero é ditada por comportamentos de exclusão ou se ambos estão apenas respondendo de forma independente aos recursos oferecidos pela matriz antrópica.

Para gerar a análise de interação multiescalar, seguimos três etapas principais no R. Primeiro, unificamos em um único banco de dados os registros de presença de cães e gatos com as métricas de paisagem calculadas para cada escala. Em seguida, aplicamos um Modelo Linear Generalizado

(GLM) para cada um dos raios de monitoramento. Diferente dos modelos anteriores, aqui a presença do gato é a variável resposta, enquanto a presença do cão e a porcentagem de uso humano entram como variáveis explicativas. Abaixo, utilizamos a lógica de ‘empacotar’ os dados por escala (função `nest`) e aplicar o modelo de forma automatizada (função `map`). O objetivo é extrair o coeficiente de inclinação (log-odds) que indica a força da relação entre as duas espécies. Se o intervalo de confiança desse coeficiente cruzar a linha do zero, concluiremos que a presença de uma espécie não dita a ocorrência da outra naquela escala específica. O código a seguir demonstra como organizar esses modelos e preparar os dados para a visualização final:

```
# 1. Preparação: Unindo dados bióticos (cães e gatos) com a paisagem
df_cooc_pre <- especies |>
  select(plot_id, calu, feca) |>
  left_join(df_pland_humano, by = "plot_id")

# 2. Modelagem com Intervalos de Confiança de 95%
cooc_robusto <- df_cooc_pre |>
  group_by(escala_buffer) |>
  nest() |>
  mutate(
    modelo = map(data, ~glm(feca ~ calu + pland_humano, data = .x, family = binomial)),
    # conf.int = TRUE calcula automaticamente os limites inferior e superior
    coefs = map(modelo, ~tidy(.x, conf.int = TRUE, conf.level = 0.95))
  ) |>
  unnest(coefs) |>
  filter(term == "calu")

# 3. Visualização
ggplot(cooc_robusto, aes(x = escala_buffer, y = estimate)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red", size = 1) +
  # geom_errorbar desenha os ICs de 95% robustos
  geom_errorbar(aes(ymin = conf.low, ymax = conf.high),
               width = 0.2, size = 0.8, color = "purple") +
  geom_point(size = 4, color = "purple") +
  labs(
    title = "Interação Biótica: Cães vs Gatos",
    subtitle = "Intervalos de Confiança (IC) de 95% calculados via verossimilhança",
    x = "Escala de Buffer (m)",
    y = "Efeito da Presença de Cães (Log-odds)"
  ) +
  theme_light(base_size = 12)
```

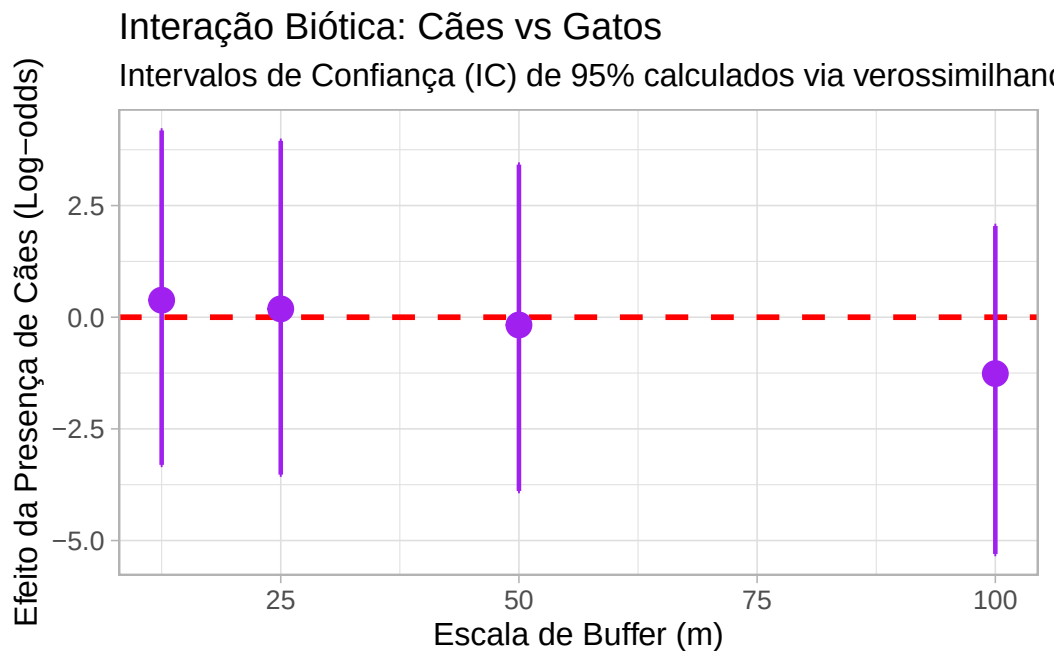


Figura 3.5: Análise multiescalar da interação biótica entre cães e gatos no Campus Marco Zero da UNIFAP, Macapá-AP. Os pontos representam as estimativas de inclinação (coeficientes Log-odds) da presença de cães sobre a probabilidade de ocorrência de gatos, controladas estatisticamente pela porcentagem de áreas de uso humano (métrica PLAND). As barras verticais indicam os intervalos de confiança (IC) de 95%. A linha tracejada vermelha no valor zero representa a hipótese nula (ausência de interação).

Em todas as escalas testadas (12,5m a 100m), os intervalos de confiança de 95% (barras roxas) cruzam a linha do zero (Figura 3.5). Isso indica que, estatisticamente, não há evidência de uma interação forte (atração ou repulsão) entre cães e gatos no Campus Marco Zero. O gráfico sugere que outros fatores (como a oferta de comida) é uma força mais poderosa na coocorrência do que a competição direta entre as duas espécies. O fato de a interação ser nula sugere que ambas as espécies podem estar respondendo de forma independente aos recursos oferecidos pelo campus (comida, abrigo em prédios), onde a disponibilidade de recursos no habitat é o fator principal que dita onde os animais estão, e não a presença um do outro.

Ao interpretarmos os resultados de coocorrência, devemos olhar para dois processos que operam simultaneamente no campus Hotspots de Recursos e Filtragem Ambiental

- Hotspots de Recursos (Atração Comum):
A ausência de segregação (intervalos cruzando o zero) sugere que a distribuição de recursos antrópicos (pontos de alimentação voluntária, lixeiras abertas e o entorno do Restaurante Universitário - RU) exerce uma força de atração superior à força de repulsão entre as espécies. Em ecologia urbana, o recurso (comida) frequentemente “força” a sobreposição de nicho espacial, mesmo entre predadores e competidores.
- Filtragem Ambiental vs. Seleção de Micro-habitat:
Enquanto o “uso humano” (PLAND) atua como um filtro macro (definindo quem consegue

viver no campus), a localização fina dos indivíduos é ditada pela oferta de subsídios energéticos. Cães e gatos na UNIFAP não estão apenas “onde há prédios”, eles estão onde há energia disponível. Se os intervalos de confiança são largos, isso indica que essa oferta de recursos é heterogênea e não foi totalmente capturada apenas pela métrica de cobertura do solo.

 Dica do Ecólogo: Por que um resultado ‘não significativo’ é valioso?

Você pode estar pensando: “Tanto código e análise para descobrir que a relação entre cães e gatos é aleatória?”. Na ciência aplicada, encontrar um **resultado não significativo** em múltiplas escalas é uma descoberta valiosa por dois motivos principais:

1. **A Independência das Espécies:** O fato de a coocorrência ser aleatória em raios de 12,5m até 100m sugere que a presença de uma espécie não é o fator que limita a outra no Campus. Isso indica que não há uma exclusão competitiva forte ou segregação espacial ativa.
2. **A Força da Matriz Urbana:** Quando a estatística aponta para a aleatoriedade biótica em várias escalas, ela está, na verdade, destacando que o **ambiente (a matriz urbana)** é uma força muito mais poderosa. Em vez de as espécies “negociarem” o espaço entre si, ambas estão respondendo de forma individual aos subsídios humanos (lixo, oferta de comida e abrigo).
3. **Saturação da Paisagem:** Em ecologia, quando duas espécies generalistas apresentam alta prevalência e coocorrência aleatória em múltiplas escalas, isso sugere que a paisagem está **saturada**. Não há barreiras ambientais ou bióticas eficazes que limitem a ocupação de cães ou gatos no campus. Ambos utilizam a matriz urbana de forma tão eficiente que a presença de um não interfere na “liberdade” de ocupação do outro.
4. **Necessidade de Gestão Proativa e Integrada:** O fato de operarem de forma independente e generalizada significa que ações isoladas (focadas em apenas uma espécie ou em pontos específicos) serão paliativas. Como não há segregação, uma **gestão proativa** é mandatória.
5. **O Valor da Evidência Multiescalar:** Confirmar que essa independência se mantém em raios de 12,5m até 100m prova que o coocorrência não é um artefato de amostragem local, mas uma característica estrutural do Campus Marco Zero. Para o gestor ambiental, isso fornece o embasamento científico para tratar a fauna doméstica não como casos isolados, mas como um componente onipresente da paisagem que exige manejo do ambiente como um todo, e não apenas o controle individual de animais.

Portanto, o “não” da estatística redireciona seu olhar: o questão não é como os animais interagem, mas sim como a paisagem urbana e ações antropicas está facilitando a presença de ambos de forma indiscriminada. Entender que o processo é **independente da escala** nos poupa de tentar resolver um problema biológico quando o desafio é, na verdade, o planejamento do ambiente.

3.3.4 Rumo ao Desenho Quase-Experimental

Para sair de uma análise puramente descritiva e caminhar para uma inferência causal mais robusta, podemos implementar um desenho quase-experimental com os dados. Um experimento verdadeiro (sortear onde haverá cães ou humanos) é impossível, mas o “quase-experimento” aproveita variações existentes.

A. Amostragem Estratificada por Gradiente de Recurso.

Implementem um desenho de Contraste de Tratamentos com dois grupos. Grupo Tratamento (Alta Oferta): pontos num raio de 50m do RU, lixeiras centrais e pontos de alimentação. Grupo Controle (Baixa Oferta): pontos de vegetação de cerrado remanescente ou mais distante pontos de alimentação. Comparem se a “Escala de Efeito” muda entre esses dois grupos. A hipótese é que, em áreas de alta oferta, a coocorrência é positiva (agregação), e em áreas de baixa oferta, a competição aparece (segregação).

B. Aproveitamento de “Experimentos Naturais” (BACI - Before-After-Control-Impact).

O campus passa por mudanças cíclicas que podem ser tratadas como tratamentos experimentais. Período de Férias vs. Período Letivo, onde a drástica redução na produção de resíduos e na circulação de pessoas no RU durante as férias é um “experimento de remoção de recurso”. Se a segregação entre cães e gatos aumentar durante as férias (quando há menos comida), vocês provam que o recurso humano é o mediador da paz (coexistência) entre as espécies. Comparar Amostragem Estratificada por Gradiente de Recurso nas épocas diferentes.

C. Mapeamento de Pontos de Alimentação como Covariável.

Adicionem uma camada de distância euclidiana até o ponto de recurso mais próximo no modelo (RU, lixeiras centrais e pontos de alimentação). No código R, em vez de usar apenas métricas da paisagem, usem `dist_recurso`. Dica Teórica: Se o AICc do modelo com `dist_recurso` for menor que o de `pland`, vocês demonstram que a ecologia das espécies domésticas na UNIFAP é guiada por pontos focais de energia e não apenas pela estrutura física da paisagem.

3.4 Conclusão

Ao longo das análises, não confiamos em apenas uma fonte, mas sim, um conjunto de evidências científicas. Usamos o pacote `cooccur` para testar interações bióticas, o `landscapemetrics` para quantificar a estrutura da paisagem e modelos lineares generalizados (GLM) para avaliar a escala de efeito. Por que isso é importante?

Rigor contra o subjetivismo: O que parece óbvio ao olhar para o mapa (ex: ‘tem cachorros perto dos gatos’) só se torna evidência científica quando resiste a testes estatísticos de diferentes naturezas. Além disso, conclusões baseadas em múltiplas análises protegem o gestor ambiental. Se você precisar propor uma política de gestão de fauna doméstica, terá em mãos não apenas um ‘acho que’, mas um corpo de evidências científicas robustas que mostra exatamente em qual escala o problema ocorre e quais fatores (bióticos ou abióticos) são irrelevantes.

Embora a ocorrência (presença-ausência) não mostre segregação, uma análise nova usando abundância relativa (frequência de fotos) poderia revelar padrões diferentes, como uma maior concentração de ambas as espécies em torno de “ilhas de recursos” (ex: Restaurante Universitário).

Avançar da Ecologia de Paisagens descritiva para o desenho quase-experimental permitira que a UNIFAP não seja apenas o local de estudo, mas um laboratório vivo onde testamos teorias sobre como subsídios humanos alteram as regras básicas das interações bióticas na Amazônia urbana.

Referências

- Dormann, C. F., J. Elith, S. Bacher, et al. 2013. “Collinearity: a review of methods to deal with it and a simulation study evaluating their performance”. *Ecography* 36 (1): 27–46. <https://doi.org/10.1111/j.1600-0587.2012.07348.x>.
- Hesselbarth, Maximilian H. K., Marco Sciaini, Kimberly A. With, Kerstin Wiegand, e Jakub Nowosad. 2019. “landscapemetrics: an open-source R tool to calculate landscape metrics”. *Ecography* 42: 1648–57. <https://doi.org/10.1111/ecog.04617>.
- Tomlin, C. D. 1990. *Geographic Information Systems and Cartographic Modeling*. Prentice Hall series em geographic information science. Prentice Hall. <https://books.google.com.br/books?id=shiAAAAAMAAJ>.